A Reproducible Benchmarking Framework for Evaluating TLS and mTLS Performance Overhead in Zero Trust-Aligned Apache Kafka Deployments

Nick Efe Oni

Supervisor: Mahshid Delavar

A report submitted in fulfilment of the requirements for the degree of MEng in Software Engineering

in the

School of Computer Science

April 23, 2026

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Nick Efe Oni

Signature: Nick Efe Oni

Date: April 23, 2026

Abstract

an abstract is provided to give a brief overview and summary of the core topics, arguments, and findings . Because it must summarize your entire project—including your final benchmarking results from Chapter 5 and your final conclusions from Chapter 7—you should write the abstract at the very end of the process, once the rest of your dissertation is completely finished.

Contents

1 Introduction **1**
1.1 Background and Motivation . . . 1
1.2 Problem Statement . . . 2
1.3 Research Aim and Objectives . . . 2
1.4 Research Question . . . 3
1.5 Artefact Overview . . . 3
1.6 Dissertation Structure . . . 4

2 Literature Review **5**
2.1 Introduction to Literature Review . . . 5
2.2 Apache Kafka & Distributed Streaming Systems . . . 6
2.3 Performance in Kafka Systems . . . 7
2.4 Security Mechanisms in Kafka . . . 9
2.5 Zero Trust Architecture . . . 11
2.6 Zero Trust in Distributed Streaming Systems . . . 12
2.7 TLS/mTLS Performance Overhead . . . 14
2.8 Benchmarking Methodologies . . . 15
2.9 Synthesis & Research Gap . . . 16

3 Design, Implementation, and Testing **19**
3.1 Chapter Overview . . . 19
3.2 Artefact Requirements . . . 19
3.3 System Architecture . . . 21
3.4 Infrastructure Provisioning . . . 23
3.5 Kafka Cluster Deployment . . . 24
3.6 Security Mode Design . . . 25
3.7 Benchmark Workload Design . . . 26
3.8 Factorial Experiment Design . . . 27
3.9 Execution and Orchestration . . . 28
3.10 Telemetry and Metric Collection . . . 31
3.11 Result Processing and Export . . . 32
3.12 Fault Tolerance and Idempotency . . . 34

3.13 Testing and Validation . . . 35
3.14 Chapter Summary . . . 37

4 Results and Discussion **38**

5 Evaluation, Conclusions, and Further Work **39**

Appendices **42**

A Ethics and GenAI Declaration **43**
A.1 Ethics Statement . . . 43
A.2 Generative AI Declaration . . . 43
A.2.1 Acknowledge . . . 43
A.2.2 Describe . . . 44
A.2.3 Evidence . . . 44
A.2.4 Declaration . . . 45

B Artefact Configuration **46**
B.1 Configuration Files . . . 46
B.2 Producer Factorial Campaign . . . 47
B.3 Security Mode Configuration . . . 48
B.4 Validity Constraints . . . 49
B.5 Consumer Validation Slice . . . 49

C Result Schema **51**
C.1 Run-Level Output . . . 51
C.2 Metadata Fields . . . 52
C.3 Parsed Result Fields . . . 53
C.4 Host Telemetry Fields . . . 53
C.5 Result-Set Output . . . 54
C.6 Metric Interpretation Caveat . . . 55

D Additional Result Tables **56**

E Repository and Reproducibility Notes **57**
E.1 Repository Structure . . . 57
E.2 Configuration-Driven Execution . . . 58
E.3 Deployment Reproducibility . . . 59
E.4 Checkpointing and Resumability . . . 59
E.5 Result Traceability . . . 60
E.6 Generated State and Version Control . . . 60
E.7 Rerunning the Artefact . . . 60
E.8 Reproducibility Limitations . . . 61

List of Figures

2.1 Simplified Apache Kafka architecture showing producer applications publishing events to a cluster of brokers, and consumer applications pulling events from the cluster. Figure generated using ChatGPT image generation from a prompt requesting a producer-broker-consumer Apache Kafka architecture diagram, April 2026. 7

2.2 Comparison of Kafka data movement under plaintext, TLS, and mTLS. Plaintext can use page-cache-backed `sendfile` zero-copy transfer, while TLS and mTLS require user-space cryptographic processing that prevents this path. 9

2.3 Kafka security data path showing how TLS/mTLS, certificate-based identity verification, principal mapping, and ACL authorisation are applied before Kafka request processing, with broker-to-broker communication also secured where configured. 11

2.4 Conceptual Zero Trust control model for a distributed streaming pipeline, illustrating how mTLS, policy enforcement, least-privilege access, and continuous monitoring are applied across producer, broker, stream-processing, and consumer components within a segmented trust boundary. 13

3.1 System architecture of the Kafka benchmarking artefact, showing local orchestration, Terraform-provisioned AWS infrastructure, private Kafka traffic, SSH control paths, broker instances, benchmark client, and result collection. 22

3.2 Benchmark execution pipeline from JSON configuration to dissertation-ready result exports. 30

3.3 Result directory and data-flow schema showing how raw per-run logs, metadata, telemetry, campaign state files, parsed results, aggregated summaries, and dissertation-ready exports are connected. 33

List of Tables

3.1 Core artefact requirements 20
3.2 Kafka transport-security modes evaluated by the artefact 26
3.3 Testing and validation evidence for the benchmarking artefact 36

A.1 Representative examples of GenAI use during the project 45

B.1 Main artefact configuration and generated plan files 47
B.2 Producer factorial campaign dimensions 48
B.3 Security modes used by the artefact 49

C.1 Run-level benchmark output files 52
C.2 Metadata field groups 52
C.3 Primary parsed result metrics 53
C.4 Host telemetry field groups 54
C.5 Result-set output files 54

E.1 Repository structure 58

Chapter 1

Introduction

1.1 Background and Motivation

Modern distributed systems increasingly rely on event-streaming platforms to connect cloud services, data pipelines, and enterprise applications [1, 2]. Apache Kafka is a widely used open-source event-streaming platform that supports high throughput, low latency, fault tolerance, and horizontal scalability [3–5]. These properties make Kafka a critical component in data-driven systems where large volumes of events must move reliably between producers, brokers, and consumers.

Kafka achieves much of its performance through partition-level parallelism, batching, append-only logs, page-cache utilisation, and zero-copy data transfer [3, 6, 7]. However, these mechanisms also make Kafka sensitive to additional processing in the data path. Security mechanisms, while optional in Kafka, are therefore essential deployment details (particularly in regard to assessing performance), as they can affect the communication and computation paths that Kafka relies on for high-throughput streaming.

At the same time, distributed systems face security risks that traditional perimeter-based models do not. In cloud and multi-service environments, internal network access can no longer be treated as inherently trustworthy, as insider threats and lateral movement attacks can originate from within the network boundary [8–10]. This has increased interest in Zero Trust Architecture, a broader system-level security paradigm, whose central principle is commonly summarised as "never trust, always verify" [8–11]. Zero Trust deployments usually emphasise strong identity, mutual authentication, least-privilege access, encrypted communication, monitoring, and policy-based control [9, 11, 12].

While Zero Trust Architecture is not a single Kafka feature, Kafka provides native mechanisms that align with several Zero Trust principles, including transport encryption, certificate-based authentication, broker-to-broker security, SASL authentication, and ACL-based authorisation [7, 13–15]. Among these, TLS and mutual TLS (mTLS) are

particularly relevant because they directly affect Kafka's data path and allow plaintext, server-authenticated, and mutually authenticated transport configurations to be compared.

1.2 Problem Statement

Existing research provides strong foundations for both Zero Trust Architecture and Apache Kafka performance. Zero Trust has been discussed in distributed and streaming systems [8,12,16], while Kafka has been widely analysed as a high-throughput event-streaming platform [4, 5, 17]. However, there is still limited systematic empirical evaluation of the performance impact of Zero Trust-aligned transport security in Kafka-based environments.

This gap matters because TLS and mTLS introduce cryptographic processing into communication paths that Kafka is designed to keep efficient. Kafka's official design documentation states that SSL/TLS prevents the use of the `sendfile` zero-copy optimisation because SSL libraries operate in user space [6]. Wider TLS literature also shows that secure transport can introduce computational and latency overhead, although the scale depends on workload, connection behaviour, and system configuration [18–20]. Therefore, the cost of TLS and mTLS cannot be assumed to be constant across Kafka deployments.

The unresolved problem is the lack of a unified, reproducible, and Kafka-specific framework for measuring this overhead under controlled configurations. In particular, there is limited evidence showing how plaintext, TLS, and mTLS compare when workload and deployment parameters are matched, and how this overhead changes across Kafka variables such as broker count, replication factor, acknowledgements, batching, compression, producer concurrency, and message size.

1.3 Research Aim and Objectives

The aim of this dissertation is to design, implement, and evaluate a reproducible benchmarking framework for quantifying the performance overhead introduced by TLS and mTLS in high-throughput Apache Kafka deployments.

The objectives are:

1. To analyse Kafka's performance mechanisms and identify how transport security may affect them.
2. To design and implement a cloud-based Kafka benchmarking artefact supporting plaintext, TLS, and mTLS.
3. To provision reproducible Kafka deployments using infrastructure-as-code and automated orchestration.

4. To execute matched benchmark workloads across security modes using explicit configuration files.
5. To measure throughput and latency for each benchmark run.
6. To collect host-level telemetry from Kafka brokers and benchmark clients, including CPU, memory, disk, and network activity.
7. To evaluate how TLS and mTLS overhead varies across Kafka workload and deployment parameters.
8. To include targeted consumer-side validation to assess whether secure transport also affects Kafka's read path.

1.4 Research Question

This dissertation addresses the following research question:

> "What is the performance overhead introduced by Zero Trust-aligned security mechanisms, particularly mTLS, in high-throughput Kafka-based distributed systems?"

1.5 Artefact Overview

To answer this research question, this dissertation designs, implements, and evaluates a configurable Kafka benchmarking artefact. The artefact provisions Apache Kafka clusters on AWS EC2 using Terraform, deploys Kafka in KRaft mode, and supports three transport-security configurations: plaintext, TLS, and mTLS. Benchmark execution is driven by structured configuration files so that equivalent workloads can be repeated across security modes and compared under matched conditions.

The main evaluation uses a producer-side factorial benchmark campaign. This campaign varies broker count, replication factor, minimum in-sync replicas, message size, target throughput, producer count, acknowledgement mode, batch size, and compression type. This allows the artefact to examine whether TLS and mTLS introduce overhead, and how that overhead changes under different Kafka workload and durability settings.

A smaller targeted consumer-side validation study is also included. This does not reproduce the full producer factorial campaign. Instead, it evaluates whether secure transport also affects Kafka's read path under selected workloads, especially where mTLS processing interacts with larger messages and compressed payloads.

For each benchmark run, the artefact records application-level metrics, including throughput

and latency, alongside host-level telemetry from the benchmark client and Kafka brokers. This includes CPU utilisation, memory utilisation, disk activity, and network activity. Raw benchmark logs are parsed into structured JSON and CSV outputs, allowing reproducible comparison of plaintext, TLS, and mTLS results.

1.6 Dissertation Structure

The remainder of this dissertation is organised as follows. Chapter 2 reviews the literature on Apache Kafka, Kafka performance, Kafka security, Zero Trust Architecture, TLS/mTLS overhead, and benchmarking methodology. It identifies the research gap addressed by this dissertation.

Chapter 3 describes the design, implementation, and testing of the Kafka benchmarking artefact. It covers the infrastructure design, Kafka deployment model, security-mode configuration, factorial workload generation, orchestration scripts, telemetry collection, result-processing pipeline, and validation strategy.

Chapter 4 presents and discusses the benchmark results. It compares plaintext, TLS, and mTLS under matched workloads and analyses the observed throughput, latency, and host-telemetry behaviour.

Chapter 5 evaluates the dissertation objectives, summarises the main findings, discusses limitations, and identifies directions for future work.

Chapter 2

Literature Review

2.1 Introduction to Literature Review

This literature review examines the areas needed to understand the performance impact of Zero Trust-aligned transport security in Apache Kafka. It does not treat Kafka and Zero Trust as separate topics. Instead, it focuses on the interaction between Kafka's high-throughput architecture and security mechanisms such as TLS and mutual TLS (mTLS), which add cryptographic processing to Kafka's communication paths.

The chapter first reviews Apache Kafka as a distributed event-streaming platform, including its producer, broker, consumer, topic, partition, and replication model. It then analyses the mechanisms that contribute to Kafka's performance, such as append-only logs, batching, compression, page-cache utilisation, partition-level parallelism, pull-based consumption, and zero-copy data transfer. These mechanisms are important because they form the baseline against which TLS and mTLS overhead must be measured.

The review then examines Kafka's native security mechanisms, including encryption, authentication, and authorisation. This leads into a discussion of Zero Trust Architecture, where a key distinction is made: Zero Trust is a system-level security paradigm, while Kafka only provides mechanisms that can support Zero Trust-aligned deployments. In this dissertation, TLS and mTLS are therefore evaluated as measurable transport-security mechanisms aligned with Zero Trust principles, rather than as a complete implementation of Zero Trust Architecture.

The chapter then reviews existing work on TLS and mTLS performance overhead, followed by benchmarking methodologies for Kafka and distributed systems. This is necessary because existing TLS studies often focus on web servers or generic client-server systems, while Kafka has a replicated, partitioned, and batch-oriented data path. Finally, the chapter synthesises these areas into the research gap addressed by this dissertation: the lack of a reproducible and Kafka-specific empirical evaluation of plaintext, TLS, and mTLS

under matched high-throughput configurations.

2.2 Apache Kafka & Distributed Streaming Systems

Distributed streaming systems, such as Apache Kafka, are designed to store, process, and move continuous streams of data across distributed applications [3, 4, 6]. Kafka is widely used because it provides fault tolerance, high availability, reliability, and high-throughput event delivery [3, 4, 17]. These properties make it suitable for cloud and enterprise systems where many applications must exchange data reliably and at scale.

Apache Kafka is built around a cluster of servers called brokers and client applications called producers and consumers [3, 4]. Producers publish events to Kafka topics, while consumers subscribe to and read those events from the brokers [3, 4]. Brokers therefore act as the intermediary layer between data-producing and data-consuming applications. A simplified version of this architecture is shown in Figure 2.1.

Kafka stores events in topics, which are implemented as append-only, time-ordered logs [3, 6, 17]. Topics are divided into partitions, and these partitions can be distributed across multiple brokers. This allows Kafka to scale horizontally because different brokers and clients can process different partitions in parallel [3, 4, 6]. Replication further improves availability and durability by storing copies of partition data across multiple brokers [3, 6].

Kafka also uses a pull-based consumption model. Instead of brokers pushing events to consumers, consumers fetch records at their own rate [6]. This decouples producers from consumers and allows consumers to process large batches without being directly overwhelmed by the production rate. This model contributes to Kafka's scalability and fault tolerance, but it also means that performance depends heavily on client behaviour, batching, partitioning, and broker configuration. These design choices create important trade-offs. Kafka performance is affected by acknowledgement settings, replication, batching, partitions, broker configuration, and client configuration [6, 7, 13, 14, 17].

Therefore, Kafka performance should not be treated as a fixed property of the platform. It is shaped by the selected configuration, deployment environment, workload, and coordination costs [6, 7, 13, 14, 17]. This is important for this dissertation because security mechanisms such as TLS and mTLS add communication and cryptographic processing to Kafka's distributed data path. Their overhead must therefore be evaluated within Kafka's architecture, rather than treated as a generic network cost [7, 13, 14, 18].

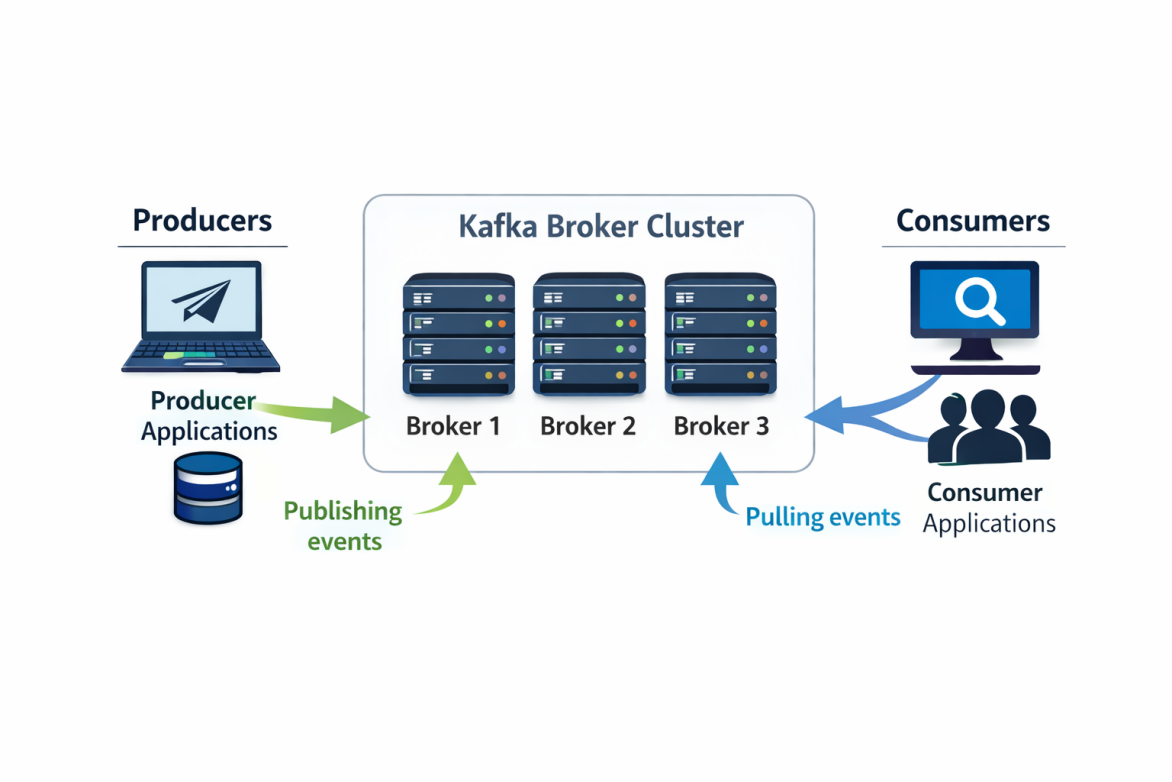

Figure 2.1: Simplified Apache Kafka architecture showing producer applications publishing events to a cluster of brokers, and consumer applications pulling events from the cluster. Figure generated using ChatGPT image generation from a prompt requesting a producer-broker-consumer Apache Kafka architecture diagram, April 2026.

2.3 Performance in Kafka Systems

Kafka performance comes from several architectural choices that reduce unnecessary data movement and allow data to be processed in parallel. Instead of relying on random-access data structures such as B-Trees, Kafka stores records in append-only logs, allowing efficient sequential writes [3, 6, 17]. Kafka also relies heavily on the operating system page cache and uses zero-copy I/O, specifically the `sendfile` system call, to transfer data from the page cache to the network socket with minimal copying [6]. These optimisations make Kafka fast, but they also make it sensitive to mechanisms that move processing back into user space, such as TLS and mTLS [6, 13, 18, 19]. This contrast is shown in Figure 2.2.

Kafka also gains performance through partitioning, batching, compression, and pull-based consumption [3, 4, 6, 17]. Topics are divided into partitions, which can be distributed across brokers and accessed in parallel by producers and consumers [3, 4, 6]. Producers can group records into larger batches, reducing network round-trip overhead and improving

throughput [6, 7, 17]. Consumers then fetch records using a pull-based model, allowing them to read batches at their own processing rate [4–6, 14].

Compression further changes Kafka's performance behaviour. By compressing batches, producers can reduce network usage and broker storage pressure [6, 7, 14]. However, this also shifts some computational cost to producers and consumers, especially during compression and decompression. This matters for this dissertation because TLS and mTLS also introduce cryptographic processing. Therefore, the combined effect of compression and transport security may depend on CPU availability, message size, and client concurrency.

Moreover, Kafka configuration choices create explicit throughput-latency-durability trade-offs [6, 7, 13, 14, 17]. For example, producers can use different acknowledgement settings. Waiting for all in-sync replicas improves durability, but can increase latency compared to weaker acknowledgement settings [6, 7, 17]. Similarly, replication factor, minimum in-sync replicas, batch size, partitioning, broker configuration, and producer concurrency can all change the balance between throughput, latency, and reliability. This means Kafka performance is not a fixed property of the platform, but an outcome of workload, deployment, and configuration choices.

This relationship can also be understood using a simple queueing-theoretic model. In publish-subscribe systems, average latency (L) can be expressed in terms of processing time (τ) and system utilisation (U) as:

$$L = \frac{\tau}{1 - U} \tag{2.1}$$

[4, 5, 19]. As processing time increases, utilisation moves closer to saturation, and latency can grow non-linearly rather than linearly [4, 5, 17, 19]. This is important because TLS and mTLS can increase processing time by adding encryption, decryption, and certificate-based authentication to Kafka communication paths [6, 13, 18, 20].

Therefore, Kafka performance must be evaluated under controlled configurations rather than assumed from general platform claims. The impact of transport security may vary with batching, compression, producer count, replication, acknowledgement mode, message size, and broker resources. This motivates the need to examine Kafka's native security mechanisms before measuring their performance impact in the artefact.

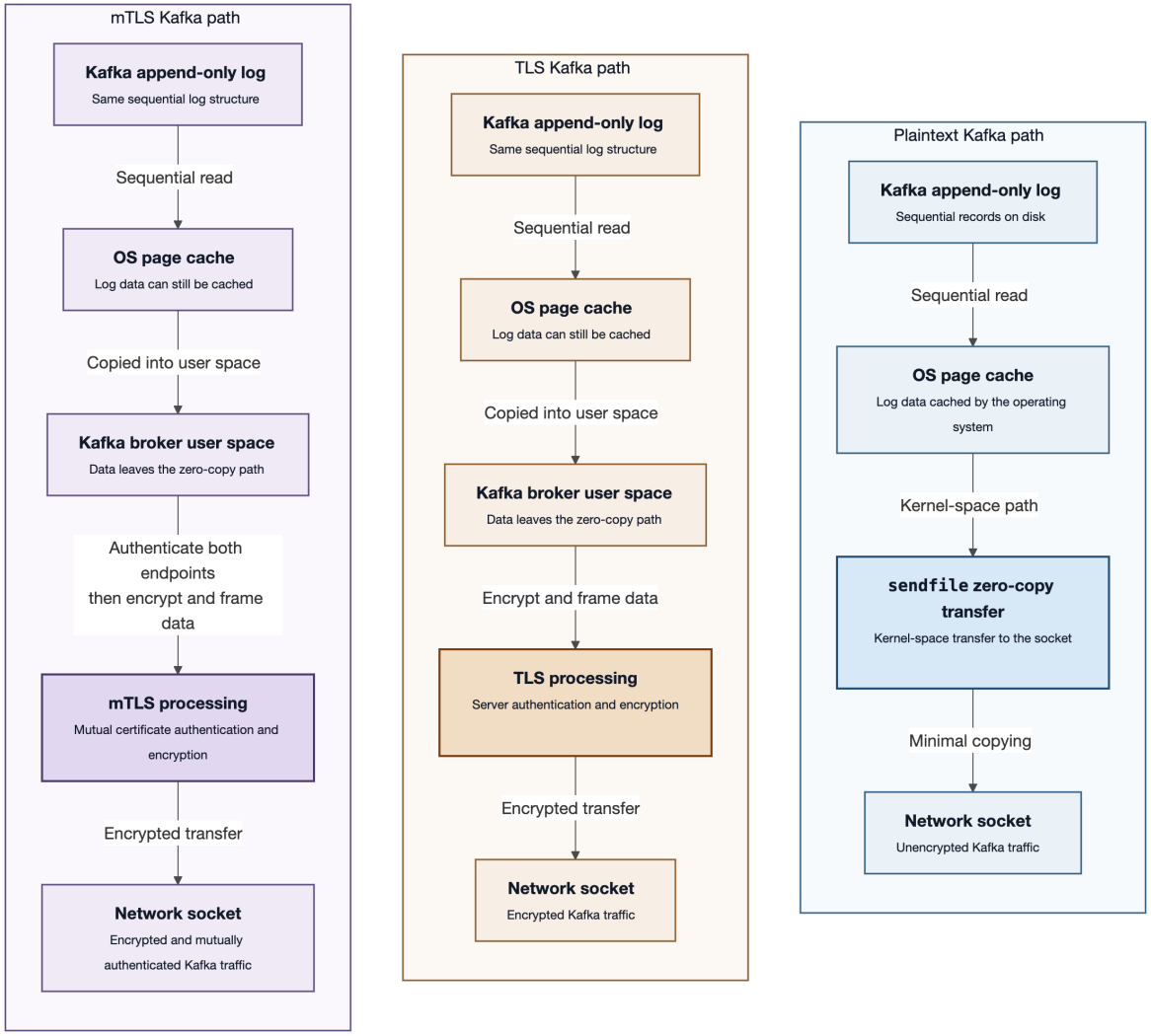

Figure 2.2: Comparison of Kafka data movement under plaintext, TLS, and mTLS. Plaintext can use page-cache-backed `sendfile` zero-copy transfer, while TLS and mTLS require user-space cryptographic processing that prevents this path.

2.4 Security Mechanisms in Kafka

Kafka provides native security mechanisms for protecting data streams, including encryption, authentication, and authorisation [3, 7, 13–15]. These mechanisms are important because Kafka is often deployed across distributed services where producers, brokers, and consumers exchange sensitive data. However, enabling security can also change Kafka's data path by adding cryptographic processing, identity verification, and access-control checks [6, 7, 13, 14].

Encryption in transit is provided through Transport Layer Security (TLS). TLS protects data

moving between Kafka clients and brokers, and can also be configured for broker-to-broker communication [7, 13, 14]. Kafka exposes this through listener and SSL configuration options, including `ssl.enabled.protocols` and `ssl.cipher.suites`, which allow operators to control the supported TLS protocols and cipher suites [7, 13, 14]. This provides confidentiality and protection against eavesdropping or man-in-the-middle attacks, but it also introduces encryption and decryption work into Kafka's communication path.

This matters for performance because Kafka's plaintext path can use efficient operating-system mechanisms such as `sendfile`. Kafka's design documentation states that this zero-copy optimisation is not used when SSL is enabled because SSL libraries operate in user space [6]. Therefore, TLS does not only add cryptographic cost; it can also change how data moves between disk, memory, and the network. This makes Kafka-specific measurement necessary, because generic TLS overhead results from web servers or simple client-server systems do not fully represent Kafka's replicated and batch-oriented architecture [17–19].

Kafka can also support mutual TLS (mTLS). In this mode, clients present certificates to brokers, and brokers verify those identities using Kafka SSL configuration such as `ssl.client.auth` [7, 13, 14]. This allows producers, consumers, and brokers to authenticate using cryptographically signed X.509 certificates. A simplified view of this security path is shown in Figure 2.3.

Moreover, mTLS is especially relevant to Zero Trust-aligned deployments because it provides encrypted communication and certificate-based identity verification. However, it can also increase the amount of cryptographic work in the system. In Kafka, this work may occur not only on client-broker communication, but also on broker-to-broker traffic when internal replication is secured [6, 13, 20]. Reconnection events can also create additional TLS or mTLS handshake load, especially when many clients reconnect at the same time [13, 18, 19].

Therefore, Kafka's native security mechanisms create a direct performance-security trade-off. They improve confidentiality, authentication, and access control, but they may also increase CPU usage, latency, and data movement through user space. For this reason, TLS and mTLS are the main focus of this dissertation. They are measurable transport-security mechanisms that align with Zero Trust principles, while also directly affecting the Kafka data path that enables high throughput.

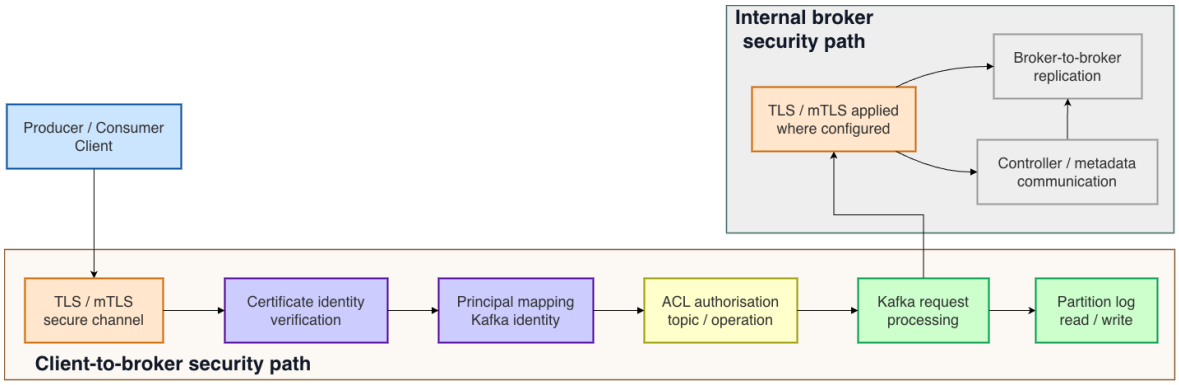

Figure 2.3: Kafka security data path showing how TLS/mTLS, certificate-based identity verification, principal mapping, and ACL authorisation are applied before Kafka request processing, with broker-to-broker communication also secured where configured.

2.5 Zero Trust Architecture

Zero Trust Architecture (ZTA) is a security model developed in response to the limits of traditional perimeter-based security [8–10, 12]. Traditional models often assume that systems inside the network boundary are trustworthy. ZTA rejects this assumption and instead follows the principle of "never trust, always verify" [8–11]. In this model, access is continuously evaluated using identity, context, policy, and device or service state, rather than being granted simply because a request originates from inside the network.

The core principles of ZTA include strong identity, least-privilege access, continuous verification, encrypted communication, monitoring, and fine-grained policy enforcement [8–11]. These principles are commonly implemented through architectural components such as Policy Decision Points (PDPs) and Policy Enforcement Points (PEPs), which separate policy decisions from policy enforcement [8, 9, 11]. ZTA is therefore not a single tool or protocol, but a system-level security approach.

Cryptographic transport security is one part of this wider model. TLS and mTLS are often used to protect communication channels and verify service identities in distributed systems [8–10, 20]. TLS provides encrypted communication and server authentication, while mTLS extends this by requiring both sides of a connection to authenticate with certificates. This makes mTLS especially relevant to Zero Trust-aligned machine-to-machine communication, where services should not be trusted purely because they are inside the same network.

However, ZTA also introduces operational and performance challenges. Implementing it can require changes to identity management, certificate handling, monitoring, network

segmentation, and access-control policy design [8–12]. In high-throughput systems, these controls can add processing and coordination overhead. This creates the key tension for this dissertation: the same mechanisms that improve trust and identity assurance may also affect the latency and throughput of systems such as Apache Kafka [6, 18, 19].

2.6 Zero Trust in Distributed Streaming Systems

The challenges of Zero Trust Architecture also apply to distributed streaming systems [1, 8–10, 12]. Modern event-streaming pipelines often span cloud services, remote devices, multi-cloud environments, and third-party networks. In these settings, components cannot be assumed to be trustworthy simply because they are inside the same network. Each producer, broker, stream processor, and consumer may need to be authenticated, authorised, encrypted, and monitored [8–10, 12]. Figure 2.4 illustrates this type of control model.

This creates a direct tension for streaming systems. These systems are designed for continuous data movement, low latency, and high throughput. However, Zero Trust-aligned controls such as mutual authentication, encryption, segmentation, and policy enforcement add work to the same communication paths that carry the data [12, 18–20]. Therefore, the main issue is not whether these controls are useful, but how their cost changes under realistic streaming workloads.

Access control is one example of this tension. Role-Based Access Control (RBAC) is widely used because it is simple to understand and operate, but it can lack the granularity required in dynamic distributed systems [8–10]. Attribute-Based Access Control (ABAC) provides finer-grained control by evaluating attributes such as identity, device state, time, location, and environmental context [8, 9, 12]. However, ABAC can be harder to configure correctly, and run-time policy evaluation can add latency if it is placed on the critical data path [8–10, 12].

Transport security creates a similar issue. Some studies suggest that modern TLS can have low steady-state overhead when connections are persistent and hardware is adequately provisioned [15, 19]. However, this does not fully resolve the problem for streaming systems. Event-streaming workloads may involve large message volumes, many clients, broker-to-broker communication, compressed batches, and reconnect events. In these cases, TLS and mTLS overhead may depend on workload shape, connection behaviour, certificate verification, and available CPU resources [12, 18–20].

This is particularly important for Apache Kafka because Kafka is designed around efficient batching, partitioning, replication, and zero-copy data transfer [6, 17]. Zero Trust-aligned transport controls, especially TLS and mTLS, can interact with these mechanisms by moving work into user space and adding cryptographic processing to

client-broker and broker-to-broker communication. Therefore, the performance impact cannot be inferred from generic TLS studies alone. It must be measured in Kafka under controlled and reproducible configurations.

This establishes the need for the empirical approach used in this dissertation. The relevant question is not whether Zero Trust principles are valuable, but what performance cost is introduced when selected Zero Trust-aligned mechanisms are applied to a high-throughput streaming platform. This motivates the comparison of plaintext, TLS, and mTLS Kafka deployments under matched workloads.

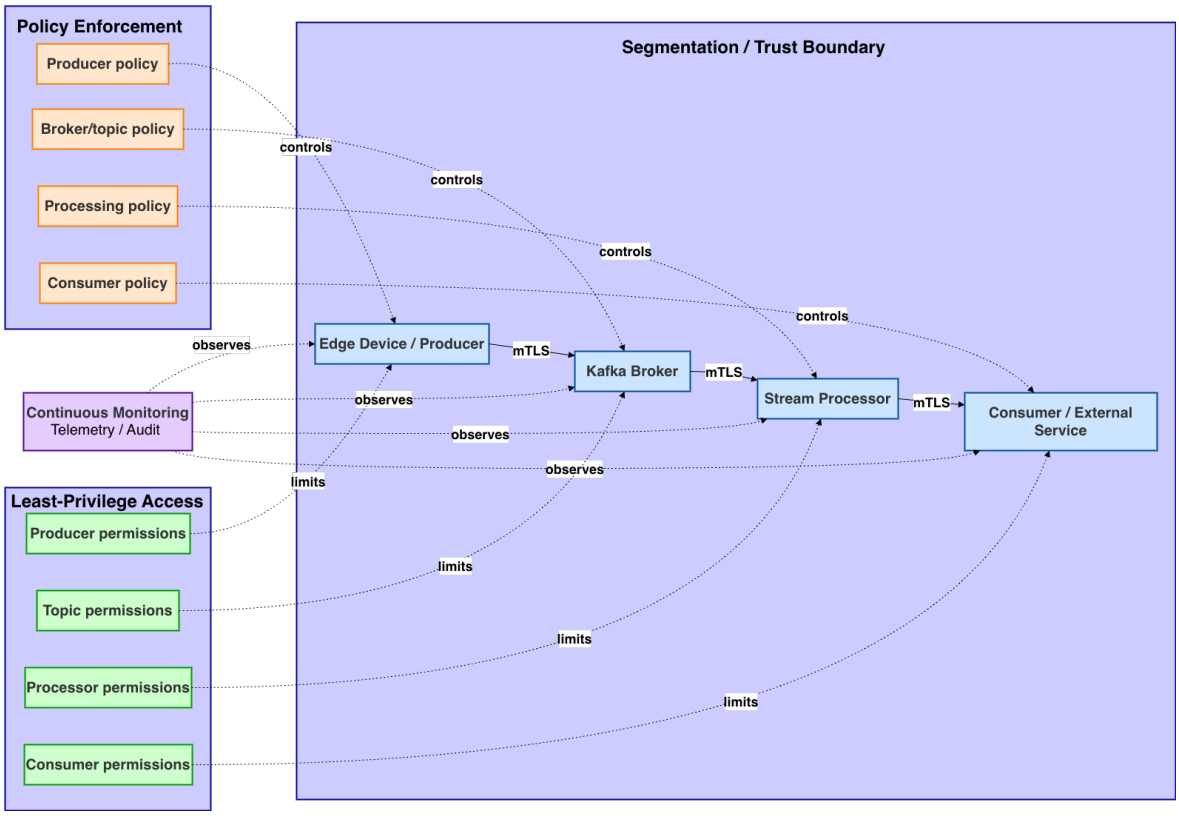

Figure 2.4: Conceptual Zero Trust control model for a distributed streaming pipeline, illustrating how mTLS, policy enforcement, least-privilege access, and continuous monitoring are applied across producer, broker, stream-processing, and consumer components within a segmented trust boundary.

2.7 TLS/mTLS Performance Overhead

Existing TLS performance studies show that encrypted transport can introduce measurable computational overhead [18,19]. Coarfa et al. separate this cost into connection setup, which uses asymmetric cryptography, and bulk data transfer, which uses symmetric encryption [18]. Their web-server experiments report a throughput degradation factor between 3.4 and 9 compared with plaintext baselines, depending on workload and session reuse [18]. They also identify RSA-based operations as a major source of overhead, consuming between 13% and 58% of web-server processing time under the evaluated workloads [18].

However, these findings cannot be transferred directly to Apache Kafka. Traditional web-server TLS benchmarks often model short-lived request-response workloads, where new connections are frequently created and where data is already handled in user space [18, 19]. Kafka is different. It is based on long-lived client-broker connections, batching, partitions, replication, and operating-system optimisations such as page-cache reuse and zero-copy I/O [6, 7, 14]. Kafka's design documentation states that `sendfile` is not used when SSL is enabled because SSL libraries operate in user space [6]. This means TLS affects Kafka not only through encryption cost, but also by changing the data-movement path.

Persistent-connection studies give a different view of TLS overhead. Iorio et al. show that persistent encrypted channels can reduce repeated handshake costs, allow TCP behaviour to stabilise, and make steady-state TLS latency small on sufficiently provisioned devices [19]. This is important because Kafka also uses long-lived connections. However, these studies still do not fully represent Kafka because they focus on generic client-server communication rather than replicated, broker-mediated streaming systems [6, 19].

In Kafka, TLS and mTLS may apply across several communication paths. These include producer-to-broker, consumer-to-broker, and broker-to-broker communication [3, 6, 13]. When mTLS is enabled, certificate-based authentication can also be required in addition to encryption [13, 20]. Therefore, the cost is not limited to a single client-server channel. It may interact with replication, batching, compression, partitioning, and broker resources [6, 7, 13, 14].

The Kafka benchmarking literature also shows a reproducibility problem. Mohammad reports that although many Kafka studies include empirical benchmarks, they often omit important configuration details such as partition count, replication factor, batch size, linger time, compression settings, and workload characteristics [17]. This makes it difficult to compare results across studies, because observed performance differences may come from configuration changes rather than security overhead alone [7, 13, 17].

Therefore, the performance impact of TLS and mTLS in Kafka should be treated as configuration-dependent. It depends on how cryptographic work is distributed across clients

and brokers, how it interacts with batching and compression, and how close the system is to CPU, network, or disk saturation [6, 7, 13, 18]. This motivates the need for a controlled benchmarking framework that compares plaintext, TLS, and mTLS under matched Kafka configurations.

2.8 Benchmarking Methodologies

Performance evaluation in distributed systems often starts with theoretical models. Network studies may use topological metrics such as bisection bandwidth or sparsest cut to estimate throughput limits [21]. Kafka studies may also use queueing models to relate latency to processing time and utilisation [5]. These models are useful because they explain why saturation can increase latency and reduce throughput. However, they are not sufficient on their own.

Jyothi et al. show that cut-based metrics do not always predict worst-case throughput, since a topology with a stronger cut metric can still perform worse under certain traffic patterns [21]. Similarly, Kafka producer evaluations show that queueing models can overestimate performance because they do not fully capture operating-system overhead, thread coordination, CPU contention, and network I/O contention [5]. Therefore, theoretical models are useful for framing the problem, but empirical benchmarking is required to measure the real behaviour of Kafka under load [6, 17].

Latency measurement also needs care. Some empirical studies rely on network-level round-trip time (RTT), using tools such as ping, traceroute, or transport-level measurements [19]. These metrics can show whether the network path is stable, but they do not capture the full application-level path. Iorio et al. argue that application-level Flow Completion Time (FCT) is more representative because it includes protocol processing, endpoint behaviour, buffering, proxying, and other effects that network-level probes can miss [19].

This matters because distributed systems can appear stable at median latency while still showing large tail-latency increases. Iorio et al. show that performance can remain similar up to high percentiles but diverge at the extreme tail due to shared infrastructure, endpoint processing, buffering, and transport behaviour [19]. Therefore, Kafka benchmarking should not rely only on average or median latency. It should also capture maximum latency and, where available, high-percentile behaviour.

Kafka benchmarking also suffers from reproducibility issues. Researchers use standardised suites such as TPCx-Kafka, synthetic frameworks such as the Yahoo Streaming Benchmark, and custom workload generators [17]. Standardised suites provide recognisable baselines, while custom workloads can model more domain-specific pipelines such as finance, IoT,

retail, or machine-learning systems [12, 15, 17]. However, Mohammad's review of 42 Kafka event-streaming studies shows that benchmark reporting is often incomplete and inconsistent [17].

This lack of transparency is a major problem for security-overhead evaluation. Kafka performance depends on parameters such as partition count, replication factor, message size, batch size, compression, producer concurrency, consumer lag, and acknowledgement settings [7, 13, 17]. If these are not reported or controlled, then performance differences may be caused by configuration variation rather than TLS or mTLS overhead. Sundbom also shows that aggregate metrics can hide internal client-side effects, such as producer queueing and outbuffer contention [5].

Therefore, evaluating Kafka transport-security overhead requires a transparent and reproducible benchmarking method. The benchmark must use explicit configuration files, matched workloads, structured output, and enough metric detail to connect application-level behaviour with resource-level causes. This motivates the artefact developed in this dissertation, which uses controlled factorial benchmark plans to compare plaintext, TLS, and mTLS under matched Kafka configurations.

2.9 Synthesis & Research Gap

The literature shows that Apache Kafka is a high-throughput distributed event-streaming platform whose performance depends on several connected mechanisms. These include append-only logs, page-cache utilisation, batching, partition-level parallelism, pull-based consumption, compression, replication, and zero-copy data transfer [3, 6, 7, 14]. These mechanisms allow Kafka to move large volumes of data efficiently, but they also make performance sensitive to configuration choices and additional processing in the data path [5–7, 13, 17]. Therefore, Kafka performance should be treated as a configurable outcome, not as a fixed platform property.

The literature also shows that modern distributed systems increasingly require security models that do not rely on implicit trust inside the network [8–10]. Zero Trust Architecture addresses this by emphasising strong identity, least-privilege access, continuous verification, encrypted communication, monitoring, and fine-grained control [8–10]. However, Zero Trust is not a single feature that can be enabled in Kafka. Instead, Kafka provides mechanisms that can support Zero Trust-aligned deployments, including TLS encryption, mTLS-based authentication, SASL authentication, ACL authorisation, and broker-to-broker security configuration [7, 13, 14].

Among these mechanisms, TLS and mTLS are the most relevant to this dissertation because they directly affect Kafka's data path. They provide a measurable basis for comparing

plaintext, server-authenticated, and mutually authenticated transport configurations [6, 13]. This creates a performance-security tension. Kafka's plaintext path can rely on efficient operating-system mechanisms such as `sendfile`, but Kafka's design documentation states that this optimisation is not used when SSL is enabled because SSL libraries operate in user space [6]. TLS and mTLS can therefore introduce both cryptographic processing and additional user-space data movement.

Existing TLS studies provide useful evidence, but they do not fully answer the Kafka-specific problem. Web-server studies show that TLS can introduce measurable overhead, especially during connection setup and asymmetric cryptographic operations [18, 20]. Persistent-connection studies show that this overhead can be smaller once handshake costs are amortised [19]. However, Kafka is not a stateless web server or a single client-server channel. It is a replicated, partitioned, batch-oriented distributed log with producer-broker, consumer-broker, and broker-to-broker communication paths [3, 6, 13]. Therefore, Kafka-specific measurement is required.

Kafka performance studies also do not fully resolve the problem. Existing work often examines throughput, batching, replication, producer parallelism, or benchmark-suite design without isolating TLS, mTLS, client authentication, and encrypted broker communication under matched configurations [5, 13, 17]. Where Kafka security is discussed, evaluations are often domain-specific, configuration-limited, or insufficiently reproducible [15, 17]. This limits the ability to compare results across workloads and deployment conditions.

The benchmarking literature further strengthens this gap. Kafka performance depends on variables such as broker count, replication factor, producer concurrency, batch size, acknowledgement mode, compression, message size, and partitioning [6, 7, 13]. If these parameters are not controlled or reported, observed differences may come from configuration variation rather than security overhead. Coarse network metrics and theoretical models are also insufficient on their own because they do not fully capture operating-system overhead, thread coordination, network I/O contention, or application-level latency [5, 19, 21]. Therefore, evaluating Kafka security overhead requires transparent, repeatable, and configuration-explicit benchmarking.

The research gap addressed by this dissertation is therefore the lack of a reproducible, systematic, and Kafka-specific benchmarking framework for quantifying the performance overhead of Zero Trust-aligned transport-security mechanisms, particularly TLS and mTLS, under controlled high-throughput configurations [6, 13, 17]. This dissertation addresses that gap by designing and evaluating a configurable Kafka benchmarking artefact that provisions cloud-based Kafka clusters, executes matched plaintext, TLS, and mTLS workloads, records producer-side throughput and latency, captures host-level telemetry from clients and brokers, and includes targeted consumer-side validation.

The contribution is both methodological and empirical. Methodologically, the artefact provides a reproducible approach for evaluating Kafka transport-security overhead under matched conditions. Empirically, it provides evidence about how TLS and mTLS affect throughput, latency, and resource utilisation across Kafka workload and deployment parameters.

Chapter 3

Design, Implementation, and Testing

3.1 Chapter Overview

This chapter describes the design, implementation, and testing of the Kafka benchmarking artefact developed to answer the research question. The artefact provides a reproducible way to measure the performance overhead introduced by Zero Trust-aligned transport-security mechanisms, particularly TLS and mTLS, in Apache Kafka deployments.

The chapter first defines the artefact requirements, then explains the system architecture, infrastructure provisioning, Kafka deployment model, security-mode design, benchmark workload design, telemetry collection, result-processing pipeline, fault-tolerance mechanisms, and testing strategy. Together, these sections show how the artefact was designed to support controlled and comparable measurement across plaintext, TLS, and mTLS configurations.

3.2 Artefact Requirements

The artefact requirements were derived from the research question and the methodological gap identified in Chapter 2. Since the dissertation evaluates Kafka transport-security overhead, the artefact must support controlled deployment, matched workload execution, structured metric capture, and repeatable comparison across security modes.

The main functional requirements are:

1. Provision cloud-based Kafka infrastructure using repeatable infrastructure-as-code.
2. Deploy Kafka in plaintext, TLS, and mTLS configurations.
3. Execute matched producer workloads across security modes.

4. Execute targeted consumer-side validation workloads.
5. Capture throughput, latency, and host-level telemetry.
6. Parse raw benchmark logs into structured result files.
7. Export results in formats suitable for analysis and dissertation reporting.

The main non-functional requirements are:

1. Reproducibility, so benchmark rows can be regenerated from explicit configuration files.
2. Comparability, so plaintext, TLS, and mTLS runs use matched workload and deployment parameters.
3. Resumability, so interrupted campaigns can continue without rerunning completed rows.
4. Fault tolerance, so transient failures do not corrupt local result state.
5. Configuration transparency, so workload and deployment parameters are recorded with each result.
6. Cost control, so cloud resources and benchmark scope remain manageable.
7. Extensibility, so additional workloads, metrics, or security modes can be added later.

Requirement	Description
Reproducibility	Benchmark runs must be generated from explicit configuration files and produce structured outputs.
Comparability	Plaintext, TLS, and mTLS runs must use matched workload and deployment parameters.
Resumability	Long-running campaigns must recover from interruption without rerunning completed rows.
Telemetry	Each run must capture application metrics and host-level resource utilisation.
Fault tolerance	Local state must remain valid when remote execution or file transfer is interrupted.

Table 3.1: Core artefact requirements

These requirements shaped the artefact design. Terraform was used for repeatable infrastructure provisioning, benchmark execution was driven by JSON configuration files, and checkpointing was added to support long-running campaigns. Host telemetry was included because throughput and latency alone are insufficient to explain whether observed overhead is caused by CPU pressure, network activity, disk activity, or client-side bottlenecks.

3.3 System Architecture

The artefact uses a cloud-based architecture built around a local workstation, Terraform infrastructure code, orchestration scripts, an AWS Virtual Private Cloud (VPC), Kafka broker instances, and a dedicated benchmark client. This design allows Kafka to be evaluated on virtual machines while also collecting host-level CPU, memory, disk, and network telemetry.

The local workstation acts as the control node. It stores the Terraform configuration, orchestration scripts, benchmark configuration files, generated plans, checkpoints, and copied result files. Terraform provisions the AWS infrastructure, while the orchestration scripts prepare the benchmark client, deploy Kafka to the brokers, configure the selected security mode, execute benchmark runs, collect telemetry, and copy results back locally.

The AWS environment contains Kafka broker EC2 instances and one benchmark client EC2 instance. The broker cluster supports both three-broker and five-broker phases, allowing broker count to be treated as an experimental variable. Kafka traffic uses private VPC addresses, while SSH orchestration uses public IP addresses. This separation keeps benchmark traffic inside the cloud network while still allowing the local workstation to manage the cluster.

The benchmark client runs the Kafka producer and consumer benchmark tools. It connects to the brokers using the listener for the selected security mode: plaintext, TLS, or mTLS. During each run, application-level benchmark output is collected alongside host telemetry from the client and brokers. Results are then copied back to the local workstation for parsing, aggregation, and export.

This architecture was selected instead of running producers directly on the broker machines because producer-side CPU and network overhead would otherwise compete with broker execution. Separating the benchmark client from the broker cluster allows client-side and broker-side telemetry to be interpreted separately. This is important for TLS and mTLS evaluation because cryptographic overhead may appear on either side of the connection.

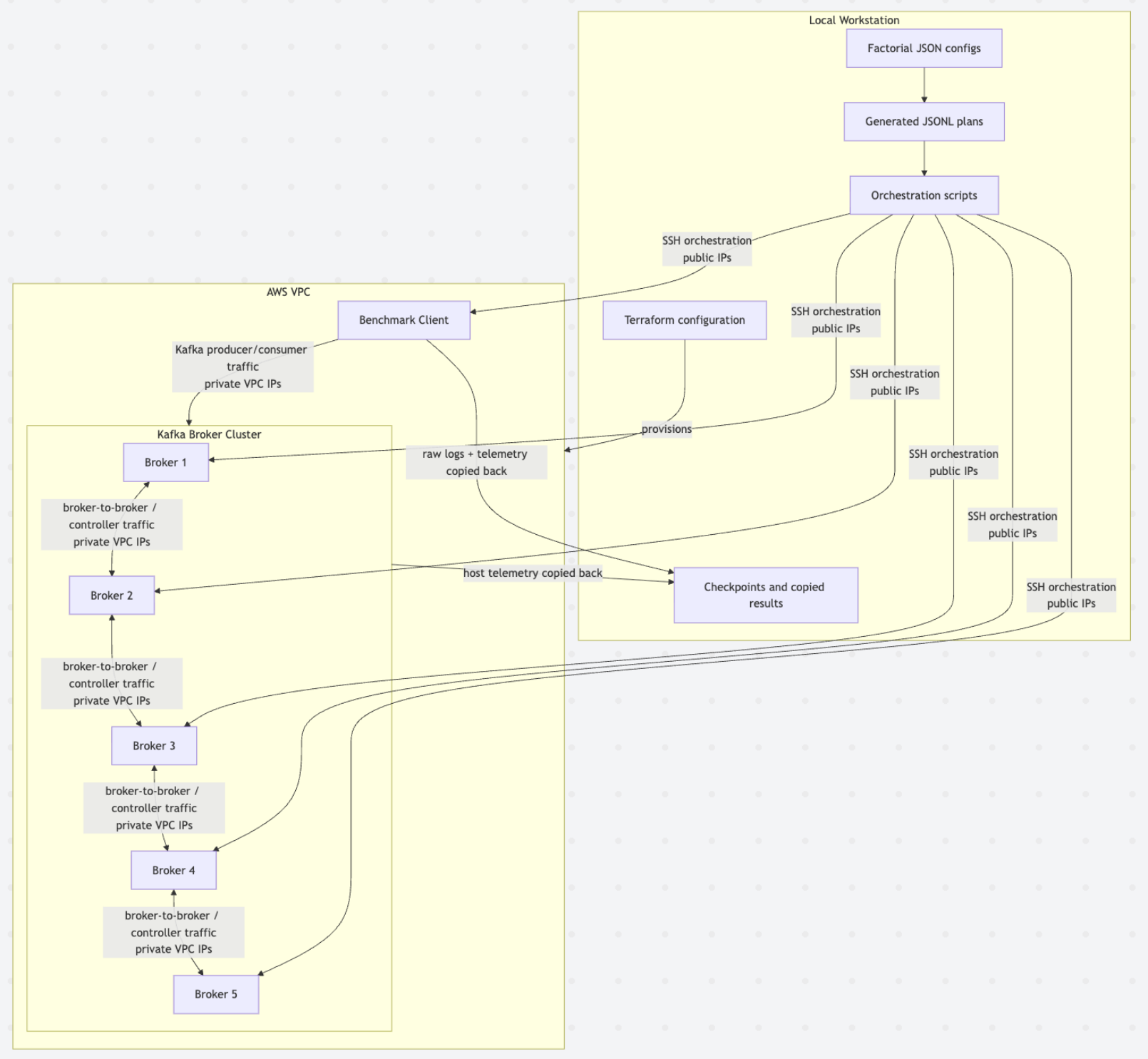

Figure 3.1: System architecture of the Kafka benchmarking artefact, showing local orchestration, Terraform-provisioned AWS infrastructure, private Kafka traffic, SSH control paths, broker instances, benchmark client, and result collection.

3.4 Infrastructure Provisioning

The infrastructure layer was implemented using Terraform and AWS EC2. Terraform was selected because it allows the cloud environment to be described as infrastructure-as-code, making the deployment repeatable and inspectable. This is important because benchmark results should be linked to a clearly defined environment rather than an informal manual setup.

The Terraform configuration provisions an AWS VPC, subnet, security groups, Kafka broker instances, and a benchmark client instance. The VPC provides an isolated network for the experiment, while security groups control SSH access, Kafka listener access, and internal cluster communication. Public IP addresses are used for SSH orchestration, while Kafka benchmark traffic uses private IP addresses inside the VPC.

The broker infrastructure supports both three-broker and five-broker phases. This prevents a three-broker run from being treated as equivalent to a five-broker deployment with unused brokers. It also allows broker count to be evaluated as a controlled deployment variable.

A separate benchmark client instance is provisioned alongside the brokers. This prevents the load generator from competing directly with broker processes on the same host. It also makes client-side CPU, memory, disk, and network activity measurable during producer and consumer runs.

Each EC2 instance uses a 40 GB gp3 root volume. This was added because early large-message benchmark runs showed that retained Kafka topic data could exhaust smaller broker disks. Larger volumes, combined with per-run topic cleanup, reduce the risk that disk exhaustion interrupts campaigns or affects benchmark validity.

Virtual machines were selected instead of containers because the experiment required control over machine-level resource behaviour and host telemetry. Containers would simplify local development, but they add another abstraction layer around CPU scheduling, network I/O, disk I/O, and memory accounting. Since this dissertation links Kafka performance to host-resource behaviour, EC2 virtual machines provide a more appropriate environment.

The main trade-off of using AWS EC2 was reduced control over noisy-neighbour effects and cloud-level variability compared with a dedicated laboratory cluster. However, EC2 was still selected because it provided repeatable provisioning, realistic cloud networking, and enough host-level visibility for the dissertation's resource-utilisation analysis. This made it more suitable than local containers for evaluating Kafka transport-security overhead.

3.5 Kafka Cluster Deployment

Kafka was deployed in KRaft mode rather than using ZooKeeper. In KRaft mode, Kafka brokers participate directly in the metadata quorum, reducing the number of external services required by the artefact. This simplifies automated deployment while reflecting the current direction of Kafka cluster management.

KRaft mode was selected because it removes the need for a separate ZooKeeper ensemble, reducing the number of moving parts in the artefact. This improves automation and makes benchmark failures easier to attribute to Kafka workload, security mode, or infrastructure behaviour rather than to an additional coordination service. The trade-off is that the evaluation focuses on modern Kafka KRaft deployments and does not compare against older ZooKeeper-based clusters.

Each broker is configured with a broker identifier and participates in the cluster. The controller quorum manages metadata such as topic definitions, partition assignments, broker membership, and leadership changes. The orchestration scripts generate the required cluster metadata, including private broker addresses and quorum configuration, before starting Kafka.

Kafka broker traffic uses private VPC IP addresses. This applies to producer, consumer, broker-to-broker, and controller communication. Public IP addresses are used only for SSH orchestration. This avoids routing Kafka data-plane traffic through the public internet and keeps communication within the intended experimental environment.

Benchmark topics are created using explicit workload parameters. Each run defines the topic, partition count, replication factor, and minimum in-sync replicas. Partition count controls parallelism, replication factor controls how many broker copies of each partition exist, and `min.insync.replicas` defines how many replicas must acknowledge writes under stronger acknowledgement settings.

Topics are created for benchmark runs and deleted after runs where appropriate. This reduces the risk that previous benchmark state affects later runs and prevents retained test data from filling broker disks during long-running campaigns.

Broker readiness is validated using Kafka API checks rather than relying only on `systemd` process status. A broker process can be running while the broker is not yet ready to serve metadata requests or client traffic. Kafka-level readiness checks therefore reduce the risk of starting benchmark workloads against a partially available cluster.

3.6 Security Mode Design

The artefact supports three Kafka transport-security modes: plaintext, TLS, and mutual TLS (mTLS). These modes allow the benchmark to compare an unsecured baseline, an encrypted server-authenticated configuration, and an encrypted mutually authenticated configuration under matched workloads.

Plaintext mode is used as the baseline. Kafka clients and brokers communicate without transport encryption or certificate-based authentication. This is not a secure production configuration, but it provides the reference point needed to measure the additional cost of TLS and mTLS.

TLS mode enables encrypted communication between Kafka clients and brokers. In this configuration, the broker presents a certificate and the client verifies the broker identity. This provides confidentiality and server authentication while measuring the overhead of encrypted transport.

mTLS mode extends TLS by requiring client certificate authentication. Clients also present certificates to the broker, allowing both sides of the connection to be authenticated. This mode is the closest benchmark configuration to a Zero Trust-aligned transport model because it combines encryption with certificate-based identity verification.

These modes do not represent a complete Zero Trust Architecture. A full Zero Trust deployment would also require broader policy enforcement, continuous monitoring, least-privilege authorisation, identity lifecycle management, and operational controls beyond Kafka transport configuration. Therefore, TLS and mTLS are evaluated as measurable Zero Trust-aligned transport-security mechanisms.

The benchmark focuses on the configured Kafka transport path used by the benchmark client and brokers. This keeps the comparison controlled, but it also means that the results should be interpreted as the overhead of the implemented Kafka security configuration rather than a complete enterprise Zero Trust deployment.

The TLS and mTLS configurations permit TLS 1.3 and TLS 1.2, with protocol negotiation handled by the Kafka/JDK SSL stack. The artefact therefore measures the deployed Kafka transport-security modes rather than isolating TLS 1.3-only or TLS 1.2-only overhead.

Mode	Description
Plaintext	Baseline with no transport encryption.
TLS	Server-authenticated encrypted transport.
mTLS	Encrypted transport with client certificate authentication.

Table 3.2: Kafka transport-security modes evaluated by the artefact

3.7 Benchmark Workload Design

The benchmark workload design contains two parts: a producer-side factorial campaign and a targeted consumer-side validation campaign. The producer campaign is the main evaluation because the research question focuses on overhead in Kafka's high-throughput write path. The consumer campaign is included as a smaller validation study to check whether secure transport also affects Kafka's read path.

The producer benchmark uses Kafka's `kafka-producer-perf-test.sh` tool. This tool was selected because it is part of the Kafka distribution and reports producer throughput and latency under configurable workload settings. The artefact extends this benchmark with structured metadata, deterministic run identifiers, concurrent producer support, host telemetry, parsing, aggregation, and checkpointed execution.

Producer concurrency is controlled through `producer_count`. When this value is greater than one, the benchmark client starts multiple producer processes for the same run. This allows the artefact to evaluate whether TLS and mTLS overhead changes under concurrent producer load.

The producer campaign varies message size, target throughput, batch size, acknowledgement setting, producer count, compression type, replication factor, minimum in-sync replicas, broker count, and security mode. These parameters were selected because they affect the throughput-latency-durability trade-off discussed in Chapter 2 and are expected to interact directly with transport-security overhead.

Message size and target throughput change the volume of encrypted data. Batch size and compression affect how much work is amortised across each network request. Acknowledgement mode, replication factor, and minimum in-sync replicas affect broker coordination and durability cost. Producer count changes client-side parallelism and can expose whether cryptographic processing becomes a client bottleneck. Broker count changes the amount of cluster capacity available for partition leadership, replication, and network handling.

Other Kafka parameters, such as retention time, log segment size, JVM options, and

operating-system tuning, were kept outside the factorial design. These parameters may affect Kafka performance, but they are less directly connected to the immediate producer-side transport-security cost and would substantially increase the campaign size. The workload design therefore prioritises parameters that are most relevant to the research question.

The consumer-side benchmark is intentionally designed as a targeted validation slice rather than a full factorial campaign. A full consumer factorial campaign would substantially increase runtime and cloud cost while duplicating much of the producer-side parameter space. The consumer slice therefore focuses on whether secure transport also affects read-path behaviour under selected workloads.

Together, the two workloads provide a balanced evaluation design. The producer campaign gives broad factorial coverage of Kafka write-path behaviour, while the consumer validation slice adds read-path evidence without making the overall experiment infeasible.

3.8 Factorial Experiment Design

The producer campaign was designed as a controlled factorial experiment. This allows the dissertation to evaluate how Kafka transport-security overhead changes across workload and deployment conditions rather than measuring only one fixed configuration.

The artefact separates experiment definition from execution. Factorial designs are described in JSON configuration files, where parameter ranges, trial counts, baseline values, and validity rules are defined. These files are expanded into JSONL execution plans, where each line represents one benchmark row with a full set of workload, deployment, and security parameters.

Each planned row is assigned a deterministic run identifier. The identifier encodes the main parameters, including security mode, broker count, replication factor, minimum in-sync replicas, message size, target throughput, batch size, acknowledgement mode, producer count, compression type, and trial number. This supports traceability because the same configuration maps to the same run identity.

Validity constraints are applied during plan generation. For example, replication factor and `min.insync.replicas` must be compatible with the selected broker count. This prevents the executor from attempting invalid Kafka configurations and ensures that comparisons are made across meaningful rows.

The final producer campaign design contains 5,184 planned rows after validity constraints are applied. This consists of 1,728 plaintext rows, 1,728 TLS rows, and 1,728 mTLS rows. The balanced structure supports matched security-mode comparison, because equivalent

workload and deployment configurations can be evaluated under each security mode.

A fully exhaustive Kafka benchmark across all producer, broker, network, JVM, and operating-system parameters would be infeasible within the time and cost limits of the dissertation. The factorial design therefore focuses on parameters most likely to affect the interaction between Kafka throughput and cryptographic overhead. This provides broader evidence than a single benchmark configuration while still keeping the campaign executable on cloud infrastructure.

Repeated trials are included because individual cloud benchmark runs can be affected by transient behaviour such as JVM warm-up, network variation, background operating-system activity, and cloud scheduling. Repetition provides a stronger basis for comparison than relying on a single run per configuration.

3.9 Execution and Orchestration

The execution layer is implemented as orchestration scripts that connect infrastructure provisioning, Kafka deployment, benchmark execution, telemetry collection, parsing, aggregation, and export. This keeps the experiment reproducible because the same scripts and configuration files can regenerate plans, resume campaigns, and rebuild summaries.

The first stage is plan generation. The `generate_factorial_plan.sh` script reads a factorial JSON configuration and expands it into a JSONL execution plan. This allows the planned campaign to be inspected, counted, filtered, and validated before cloud execution begins.

The main execution script is `run_factorial_plan.sh`. This script reads the JSONL plan, applies filters such as security mode or broker count, and executes rows that have not already completed. For each row, it prepares metadata, creates the benchmark topic, starts telemetry, runs the producer or consumer benchmark command on the benchmark client, copies outputs back locally, parses the result, and records completion state.

Campaign state is recorded using `started.jsonl`, `completed.jsonl`, `failures.jsonl` where applicable, and a checkpoint file. This matters because large campaigns may be interrupted by SSH timeouts, local process termination, cloud instability, or deliberate pauses. On resume, completed rows are skipped rather than rerun.

After each run, parser scripts convert raw Kafka benchmark logs and telemetry files into structured result data. The per-run output includes raw logs, run metadata, host telemetry, and a parsed `result.json`. Aggregation scripts then combine valid per-run results into `summary.json` and `summary.csv`. Export scripts produce dissertation-ready

CSV tables, LaTeX tables, and SVG plots where required.

This orchestration design supports auditability and reproducibility. Each run has explicit metadata, each campaign has recorded state, and each exported result can be traced back to raw benchmark output.

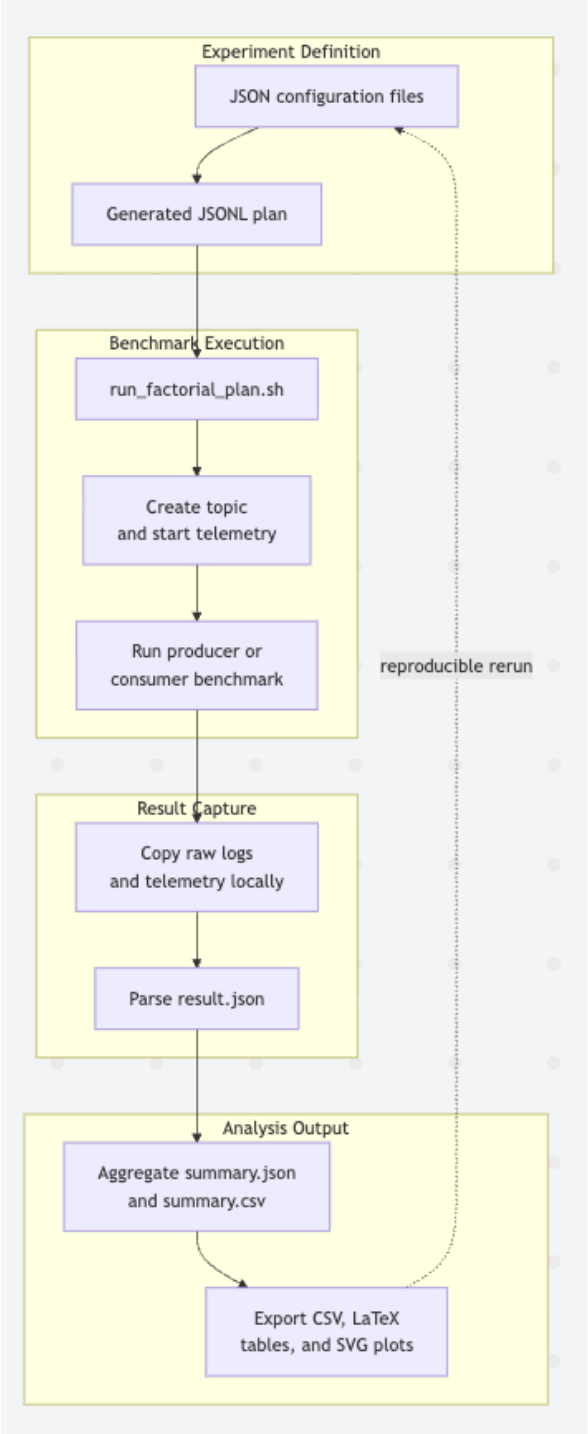

Figure 3.2: Benchmark execution pipeline from JSON configuration to dissertation-ready result exports.

3.10 Telemetry and Metric Collection

The artefact collects both application-level metrics and host-level telemetry. Throughput and latency show the benchmark outcome, but they do not explain why a run behaves differently under plaintext, TLS, or mTLS. Host telemetry provides additional evidence about CPU, memory, network, and disk behaviour.

Application-level metrics are collected from the Kafka benchmark tools. For producer runs, the main metrics are throughput in records per second, throughput in MB per second, average latency, and maximum latency. For consumer runs, the parsed output records consumer throughput and read-path metrics where available.

Host telemetry is collected from the benchmark client and Kafka broker instances during benchmark execution. The telemetry includes CPU utilisation, memory utilisation, network receive/transmit activity, and disk read/write sector counters. Collecting telemetry from both client and brokers is important because TLS and mTLS overhead may appear on either side of the connection.

Host telemetry was included to reduce the risk of misattributing performance degradation. For example, a throughput reduction under mTLS could be caused by broker CPU saturation, client-side encryption cost, network pressure, disk activity, or a combination of these factors. Collecting telemetry from both brokers and the benchmark client makes these interpretations more defensible. However, the telemetry is still host-level rather than JVM-internal, so it cannot directly attribute overhead to specific Kafka threads or SSL library calls.

Each run also records workload and deployment metadata, including run identifier, security mode, broker count, replication factor, minimum in-sync replicas, message size, target throughput, batch size, acknowledgement mode, producer count, compression type, trial number, and topic configuration. This metadata allows results to be grouped and compared under matched conditions.

Some result files include interval-derived p95 and p99 fields. These are treated as diagnostic indicators rather than true per-record latency percentiles, because they are derived from interval-level benchmark output rather than a full per-record latency distribution.

The combined metric set therefore supports both comparison and explanation. Throughput and latency measure the application-level effect of each security mode, while telemetry helps relate those effects to resource utilisation.

3.11 Result Processing and Export

The artefact separates raw execution from result processing. Each run produces a directory containing raw Kafka benchmark output, metadata, telemetry files, and parsed result data. This preserves the original evidence while also providing structured outputs for analysis.

Raw benchmark logs are retained so that parsed results can be checked against the original Kafka tool output. For producer runs, this includes the raw producer performance log. For consumer runs, this includes the consumer benchmark output. Topic creation and cleanup logs are also retained where available.

Raw logs were retained because parsed summaries can hide parser errors or edge cases in benchmark output. Keeping the original logs allows summary values to be checked against the source output and allows the parser to be rerun if the result schema changes. This improves reproducibility because the dissertation does not depend only on final CSV files.

Each run contains a `metadata.json` file recording the workload, deployment, and security parameters. The parser combines this metadata with raw benchmark output to produce `result.json`, which acts as the canonical structured result for that run.

At campaign level, aggregation scripts combine valid `result.json` files into `summary.json` and `summary.csv`. The JSON summary preserves structured fields, while the CSV summary provides a flat table for analysis, plotting, and statistical comparison.

The export layer produces dissertation-ready outputs from aggregated summaries. These include CSV tables, LaTeX tables, SVG plots, and security comparison exports. This design supports transparency because raw logs, per-run structured results, campaign summaries, and final exports are all retained as separate stages.

Figure 3.3 shows how raw benchmark evidence is transformed into structured outputs. Each benchmark run creates a deterministic result directory containing metadata, raw benchmark logs, host telemetry, topic logs, and a parsed `result.json`. Campaign-level state files then record which rows were started, completed, failed, or checkpointed. The aggregation stage combines valid per-run results into `summary.json` and `summary.csv`, which are then used to create comparison tables and plots for the dissertation.

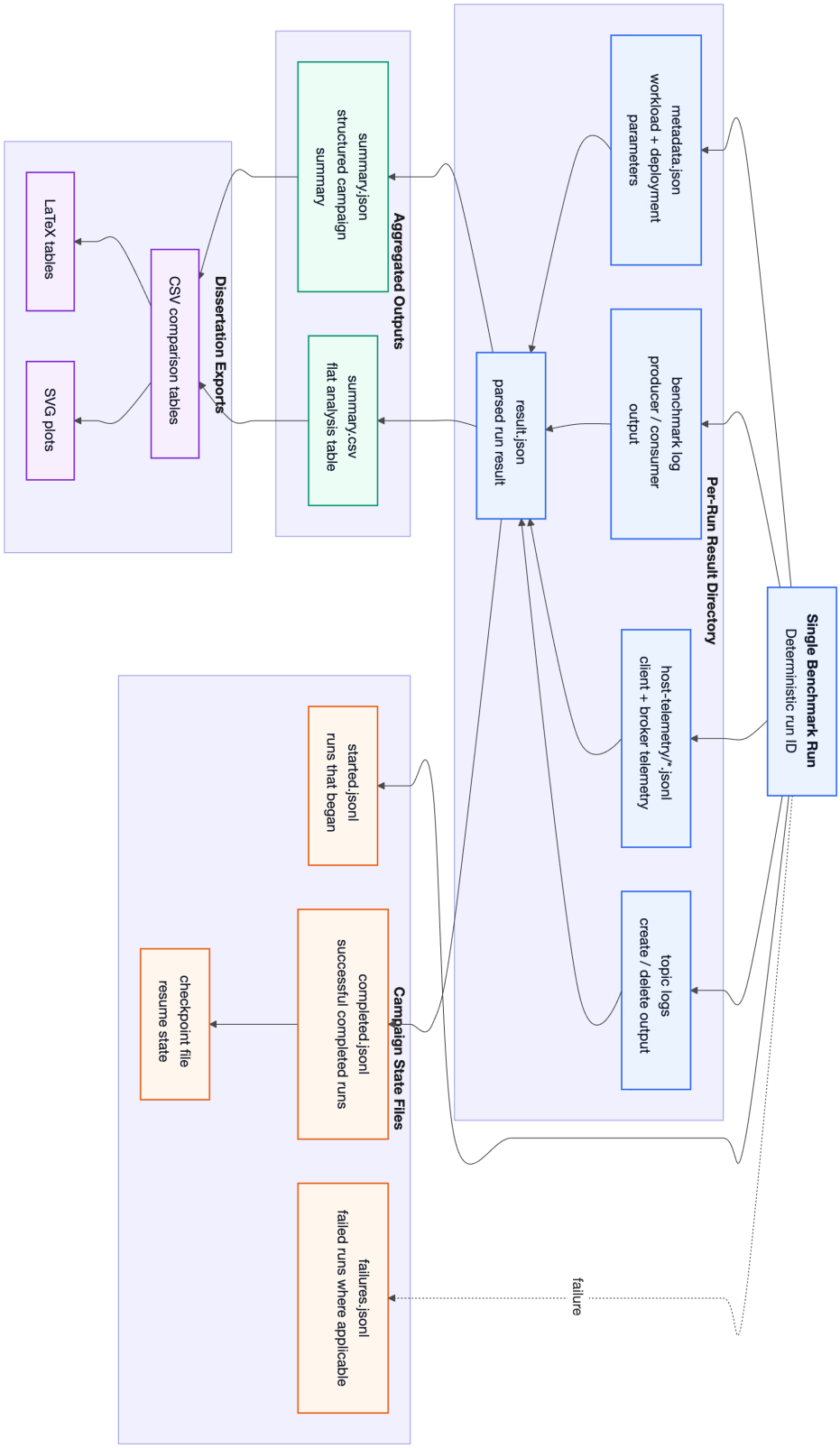

Figure 3.3: Result directory and data-flow schema showing how raw per-run logs, metadata, telemetry, campaign state files, parsed results, aggregated summaries, and dissertation-ready exports are connected.

3.12 Fault Tolerance and Idempotency

Fault tolerance and idempotency were important because the campaigns are long-running and executed on remote cloud infrastructure. Execution can be interrupted by SSH failures, local process termination, network instability, benchmark errors, or deliberate pauses to control cost.

Deployment checkpointing records which preparation stages have already completed, such as inventory generation, benchmark client preparation, broker bootstrap, Kafka configuration, and readiness validation. If deployment is interrupted, it can resume from recorded state rather than starting again from the beginning.

Benchmark checkpointing records row-level execution state. The executor writes `started.jsonl`, `completed.jsonl`, `failures.jsonl` where applicable, and a checkpoint file. Before starting a run, it checks whether the deterministic run identifier has already completed. If it has, the row is skipped.

A row is treated as complete only after the remote benchmark finishes, result files are copied back, raw output is parsed, and the checkpoint is updated. This distinction prevents a started but interrupted row from being treated as valid.

Local writes and result copying are handled through temporary files or temporary directories where possible. The completed output is moved into its final location only after copying or parsing succeeds. This reduces the risk that an interrupted transfer leaves incomplete results marked as valid.

Topic cleanup is also part of the fault-tolerance design. Benchmark topics are deleted after runs where appropriate so that retained data does not fill broker disks. This was particularly important for large-message workloads.

These mechanisms are not only engineering conveniences. They protect benchmark validity by preventing partial runs, duplicate completed rows, stale topic state, or interrupted file transfers from being treated as clean experimental results.

Together, deployment checkpoints, benchmark checkpoints, deterministic run identifiers, temporary result copying, retry wrappers, topic cleanup, and Kafka API readiness checks make the artefact resilient to interruption and suitable for repeated execution.

3.13 Testing and Validation

Testing was carried out at multiple levels because the artefact combines infrastructure provisioning, Kafka deployment, security configuration, benchmark execution, telemetry capture, and result processing. The purpose was to validate the full benchmarking pipeline, not only individual scripts.

Infrastructure testing checked whether Terraform could provision the required AWS resources, including the VPC, subnet, security groups, Kafka broker instances, benchmark client instance, and storage volumes. Kafka deployment testing checked broker availability using Kafka API-level readiness checks.

Security-mode testing used smoke tests for plaintext, TLS, and mTLS. The plaintext smoke test validated the baseline producer path. The TLS smoke test validated encrypted server-authenticated transport. The mTLS smoke test validated encrypted transport with client-certificate authentication.

Consumer-side testing validated the read-path benchmark. The consumer smoke test confirmed that benchmark topics could be populated and consumed successfully under the selected security mode. The targeted mTLS consumer validation slice also confirmed that consumer-side execution, telemetry capture, and structured result processing worked together.

Resumability was tested by restarting checkpointed campaigns and confirming that completed rows were skipped. Heavy-row testing was used to validate larger message sizes and concurrent producer configurations. Parser and aggregation testing confirmed that raw Kafka logs could be converted into `result.json`, `summary.json`, and `summary.csv`.

Test type	Purpose	Evidence produced
Terraform validation	Confirm infrastructure provisioning.	EC2 instances, VPC, subnet, security groups, and storage volumes were provisioned successfully.
Broker readiness checks	Confirm Kafka API availability before benchmark execution.	Kafka metadata and topic operations succeeded before benchmark workloads started.
Plaintext smoke test	Validate the baseline producer path.	Producer benchmark completed and produced parsed result output.
TLS smoke test	Validate encrypted `server-authenticated` transport.	TLS client configuration connected successfully and completed benchmark execution.
mTLS smoke test	Validate client-certificate authentication.	mTLS client certificate authentication completed successfully.
Consumer smoke test	Validate the read-path benchmark.	Consumer workload completed and produced structured output.
Resume test	Confirm checkpointed execution skips completed rows.	Previously completed run identifiers were skipped on resume.
Heavy-row test	Confirm larger workloads can execute.	Large-message and concurrent-producer rows completed without invalidating campaign state.
Parser test	Confirm structured result generation.	Raw logs were converted into `result.json`, `summary.json`, and `summary.csv`.

Table 3.3: Testing and validation evidence for the benchmarking artefact

The testing activities produced concrete validation evidence across the benchmark pipeline. Terraform validation confirmed that the AWS infrastructure could be provisioned repeatedly. Kafka API readiness checks confirmed that brokers were able to respond to Kafka operations before benchmark execution began. Plaintext, TLS, and mTLS smoke tests each completed producer benchmark runs successfully, confirming that the benchmark client could connect using the expected listener and security configuration.

The consumer validation slice also completed successfully, confirming that the read path could execute with secure transport, telemetry capture, and structured result parsing. The producer campaigns completed using checkpointed execution, with interrupted rows recoverable through the checkpoint mechanism. Completed rows were recorded in `completed.jsonl`, failed rows were separated into `failures.jsonl` where applicable, and parsed outputs were written into structured result files. Final campaign counts and performance results are discussed in Chapter 4.

3.14 Chapter Summary

This chapter described the design, implementation, and testing of the Kafka benchmarking artefact. The artefact was designed around reproducibility, comparability, resumability, telemetry, and configuration transparency. These requirements shaped the use of Terraform-based AWS provisioning, EC2 broker and benchmark client instances, Kafka KRaft deployment, explicit workload configuration, host telemetry, checkpointed orchestration, and structured result processing.

The chapter also described how plaintext, TLS, and mTLS modes are implemented and compared. Plaintext provides the baseline, TLS adds encrypted server-authenticated transport, and mTLS extends this with client certificate authentication. These modes are evaluated as Zero Trust-aligned transport-security mechanisms rather than as a complete Zero Trust Architecture.

The final design is not only a collection of benchmark scripts, but a controlled experimental artefact. It defines the infrastructure, workload parameters, security modes, result schema, telemetry collection, checkpointing behaviour, and validation process needed to compare Kafka transport-security overhead under matched conditions.

The following chapter uses the outputs of this artefact to evaluate the performance overhead of TLS and mTLS under matched Kafka workloads.

Chapter 4

Results and Discussion

1. A clear results overview explaining what was executed and what was excluded. 2. A table of completed runs by security mode, broker count, failures, and valid rows. 3. Matched plaintext/TLS/mTLS comparison tables. 4. Throughput comparison plots. 5. Average/max latency comparison plots. 6. Host telemetry analysis: client CPU, broker CPU, network TX/RX, memory, disk. 7. Factor interaction discussion: message size, producer count, acks, compression, replication factor. 8. Careful caveats about p95/p99 if derived from interval summaries. 9. Direct answer to the research question.

Chapter 4: completed run counts table Chapter 4: matched security comparison summary table Chapter 4: top overhead cases table

Chapter 5

Evaluation, Conclusions, and Further Work

Objective 1: Kafka mechanisms analysed. Objective 2: Cloud artefact implemented. Objective 3: plaintext/TLS/mTLS deployments supported. Objective 4: matched workloads executed. Objective 5: throughput/latency captured. Objective 6: telemetry captured. Objective 7: overhead evaluated across parameters. Objective 8: consumer validation completed.

Chapter 5: objectives evaluation table

Bibliography

[1] A. Kumar, "Distributed systems in modern enterprise architecture: Challenges and solutions," *IJSAT*, vol. 16, no. 1, p. 2447, 2025.

[2] "Powered by," https://kafka.apache.org/powered-by/, accessed: 2026-03-24.

[3] "Introduction," https://kafka.apache.org/42/getting-started/introduction/, accessed: 2026-03-24.

[4] T. P. Raptis and A. Passarella, "A survey on networked data streaming with apache kafka," *IEEE Access*, vol. 11, pp. 85 333–85 350, 2023.

[5] A. Sundbom, "Parallel kafka producer applications: Their performance and its limitations," Master's thesis, KTH Royal Institute of Technology, 2023.

[6] "Design," https://kafka.apache.org/42/design/design/, accessed: 2026-03-24.

[7] "Producer configs," https://kafka.apache.org/42/configuration/producer-configs/, accessed: 2026-03-26.

[8] H. Kang, G. Liu, Q. Wang, L. Meng, and J. Liu, "Theory and application of zero trust security: A brief survey," *Entropy*, vol. 25, no. 12, p. 1595, 2023.

[9] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, "Zero trust architecture (zta): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57 143–57 179, 2022.

[10] V. A. Vora, "Demystifying zero trust security: The no-trust network paradigm," *Journal of Computer Science and Technology Studies*, vol. 7, no. 3, pp. 141–148, 2025.

[11] M. Tsai, S. Lee, and S. W. Shieh, "Strategy for implementing of zero trust architecture," *IEEE Transactions on Reliability*, vol. 73, no. 1, pp. 93–100, 2024.

[12] H. Bhoite, "Zero-trust architecture in streaming dataflows," https://www.authorea.com/doi/full/10.36227/techrxiv.175303721.12297807, accessed: 2026-03-26.

[13] "Broker configs," https://kafka.apache.org/42/configuration/broker-configs/, accessed: 2026-03-26.

[14] "Consumer configs," https://kafka.apache.org/42/configuration/consumer-configs/, accessed: 2026-03-26.

[15] J. Meka, "Financial services cloud transformation: Securing sensitive data in kafka event streams," *Journal of Computer Science and Technology Studies*, vol. 7, no. 4, pp. 1023–1028, 2025.

[16] A. A. Süzen and O. Ceylan, "The advantages and implementation challenges within the scope of the basic principles of transition to zero trust architecture," *International Journal of 3D Printing Technologies and Digital Industry*, vol. 8, no. 3, pp. 416–427, 2024.

[17] M. Mohammad, "Analysis of design patterns and benchmark practices in apache kafka event-streaming systems," in *2025 5th International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, 2025, pp. 612–617.

[18] C. Coarfa, P. Druschel, and D. S. Wallach, "Performance analysis of tls web servers," *ACM Transactions on Computer Systems*, vol. 24, no. 1, pp. 39–69, 2006.

[19] M. Iorio, F. Risso, and C. Casetti, "When latency matters: Measurements and lessons learned," *SIGCOMM Computer Communication Review*, vol. 51, no. 4, pp. 2–13, 2021.

[20] C. H. Jack, S. K. Teck, L. T. Ming, and D. Y. Hong, "An overview analysis of authentication mechanism in microservices-based software architecture: A discussion paper," in *2023 IEEE 8th International Conference on Software Engineering and Computer Systems (ICSECS)*, 2023, pp. 1–6.

[21] S. A. Jyothi, A. Singla, P. B. Godfrey, and A. Kolla, "Measuring and understanding throughput of network topologies," in *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2016, pp. 761–772.

Appendices

Appendix A

Ethics and GenAI Declaration

A.1 Ethics Statement

This dissertation uses a computer-based experimental methodology. The artefact provisions Apache Kafka clusters on AWS EC2, executes synthetic benchmark workloads, and records application-level performance metrics and host-level telemetry. The project does not involve human participants, user studies, interviews, surveys, behavioural observation, or the collection of personal data.

The benchmark data used in this dissertation is generated synthetically by Kafka producer and consumer tools. The recorded data consists of technical machine metrics, including throughput, latency, CPU utilisation, memory utilisation, disk activity, network activity, workload metadata, and benchmark execution status. No personally identifiable information is collected, processed, or analysed.

The project therefore does not require ethical approval for human-participant research. The main ethical and professional considerations relate to responsible cloud-resource usage, secure handling of credentials, and accurate reporting of experimental limitations. AWS credentials, SSH keys, certificates, and generated security material are excluded from version control where appropriate.

A.2 Generative AI Declaration

A.2.1 Acknowledge

Generative AI tools were used as support tools during this project. Their use included assistance with drafting and refining dissertation wording, reviewing structure against university guidance, improving clarity of technical explanations, and supporting software development planning. Generative AI was also used to create the initial Apache Kafka architecture diagram used in Chapter 2.

The project design, artefact implementation, benchmark execution, result interpretation, and final academic judgement remain the author's own work. AI-generated material was reviewed, edited, and adapted before inclusion.

A.2.2 Describe

Generative AI was used mainly for support and review rather than as a source of evidence. Its use included:

- suggesting clearer wording for dissertation sections;
- helping restructure paragraphs and chapter sections;
- reviewing whether sections aligned with the university marking criteria;
- helping explain technical trade-offs such as virtual machines versus containers;
- assisting with implementation planning and debugging strategy for the artefact;
- generating an initial Apache Kafka architecture diagram, which was then reviewed for relevance to the dissertation.

Generative AI was not used as an independent factual source. Technical claims were checked against cited literature, official Apache Kafka documentation, the artefact source code, and benchmark results. The author remains responsible for the accuracy, originality, and final form of all submitted work.

A.2.3 Evidence

Representative examples of GenAI use are summarised below.

Use case	Example prompt summary	How the output was used
Dissertation structure review	Asked for Chapter 1 and Chapter 2 to be reviewed against university marking criteria.	Used to identify missing structure, such as explicit aims, objectives, and clearer research gap framing.
Chapter wording support	Asked for sections to be rewritten in a clearer academic style while preserving the author's tone.	Outputs were edited and adapted before inclusion.
Artefact explanation	Asked for help explaining the Kafka benchmarking artefact, including infrastructure, security modes, telemetry, and result processing.	Used as drafting support for Chapter 3, checked against the implemented repository.
Technical planning	Asked for advice on benchmark design, fault tolerance, idempotency, and telemetry collection.	Used to support development planning; final implementation was tested directly.
Figure generation	Prompt requested a simplified Apache Kafka architecture diagram showing producers, brokers, topics, and consumers.	Output was used as Figure 2.1 and disclosed in the figure caption.

Table A.1: Representative examples of GenAI use during the project

A.2.4 Declaration

The author confirms that no content created by generative AI technologies has been submitted without review, adaptation, and acknowledgement. The author remains responsible for the dissertation argument, artefact implementation, experimental execution, result interpretation, and final submitted content.

Appendix B

Artefact Configuration

This appendix summarises the configuration structure used by the benchmarking artefact. The artefact is driven by explicit configuration files rather than hard-coded experiment settings. This was done to make the benchmark design repeatable, transparent, and easier to compare across plaintext, TLS, and mTLS modes.

The main configuration files are stored under `config/`. Baseline files define fixed workload defaults, while factorial files define the larger parameter spaces used for the final evaluation. Generated execution plans are written as JSONL files under `.orchestration/`, where each line represents one benchmark row. This separates the experiment design from the execution logic and allows the same plan structure to be resumed after interruption.

B.1 Configuration Files

The artefact uses three main categories of configuration file. Baseline configuration files describe default workload settings. Sweep configuration files describe one-factor-at-a-time exploratory experiments. Factorial configuration files describe the final multi-parameter experiment spaces used for the security-overhead evaluation.

The final producer campaign is defined by `config/factorials/security-overhead-final.json`. The targeted consumer validation study is defined by `config/factorials/consumer-security-slice.json`. These files are converted into JSONL execution plans before running. The generated producer plan is stored as `.orchestration/security-overhead-final-plan.jsonl`, while the generated consumer plan is stored as `.orchestration/consumer-security-slice-plan.jsonl`.

Temporary deployment metadata, generated plans, checkpoints, inventory files, and Terraform outputs are also stored under `.orchestration/`. This directory is not part of the permanent artefact configuration because it contains environment-specific state, including generated cluster metadata and credentials. It is therefore excluded from version control

where appropriate.

The main configuration files used by the artefact are listed in Table B.1. The long paths are shown explicitly because they form part of the reproducibility trail for the artefact.

Configuration path	Purpose
`config/baselines/plaintext-default.json`	Defines the original plaintext baseline workload settings.
`config/baselines/plaintext-requested-baseline.json`	Defines the requested plaintext baseline used during expanded benchmark development.
`config/factorials/security-overhead-final.json`	Defines the final producer-side factorial campaign across plaintext, TLS, and mTLS modes.
`config/factorials/consumer-security-slice.json`	Defines the targeted consumer-side validation slice.
`.orchestration/security-overhead-final-plan.jsonl`	Generated JSONL execution plan for the final producer campaign.
`.orchestration/consumer-security-slice-plan.jsonl`	Generated JSONL execution plan for the consumer validation slice.
`.orchestration/security-overhead-final-mtls-broker5.checkpoint`	Example checkpoint file used to resume the mTLS five-broker producer campaign.
`.orchestration/consumer-security-slice-mtls-broker5.checkpoint`	Example checkpoint file used to resume the mTLS five-broker consumer slice.

Table B.1: Main artefact configuration and generated plan files

B.2 Producer Factorial Campaign

The main evaluation is based on a producer-side factorial campaign. This campaign is used because the research question requires more than a single plaintext, TLS, and mTLS comparison. Kafka performance depends on workload and deployment configuration, so the security overhead must be measured across a controlled range of Kafka-relevant parameters.

The final producer campaign design contains 5,184 planned rows after validity constraints are applied. These rows are divided evenly across the three security modes, with 1,728 planned rows each for plaintext, TLS, and mTLS. The same workload dimensions are used across the security modes so that comparisons can be made under matched conditions.

The campaign varies broker count, replication factor, minimum in-sync replicas, message size, target throughput, batch size, acknowledgement setting, producer count, compression type, and trial index. These variables were selected because they directly influence Kafka's throughput-latency-durability trade-off and therefore may affect how visible TLS or mTLS overhead becomes.

The generated producer plan is stored as `.orchestration/security-overhead-final-plan.jsonl`. Each line in this file represents one planned benchmark row. The executor reads this plan, applies runtime filters such as security mode and broker count, skips rows already recorded in the checkpoint, and writes completed rows into the relevant result set.

Dimension	Role in the experiment
Security mode	Compares plaintext, TLS, and mTLS transport configurations.
Broker count	Tests whether cluster size changes the observed overhead.
Replication factor	Controls durability and replication cost.
Minimum in-sync replicas	Controls how many replicas must acknowledge writes.
Message size	Tests whether larger payloads amplify encryption and data-movement costs.
Target throughput	Controls the offered producer load.
Batch size	Controls producer batching and amortisation of network overhead.
Acknowledgement mode	Changes the durability and latency behaviour of producer writes.
Producer count	Controls producer-side concurrency.
Compression type	Tests whether compression interacts with secure transport overhead.
Trial index	Provides repeated measurements for the same configuration.

Table B.2: Producer factorial campaign dimensions

B.3 Security Mode Configuration

The artefact uses three security modes. Plaintext provides the baseline with no transport encryption. TLS enables encrypted transport with server authentication. mTLS extends TLS by requiring certificate-based client authentication as well as encrypted communication.

These modes do not represent a complete Zero Trust Architecture. Instead, TLS and mTLS are evaluated as Zero Trust-aligned transport-security mechanisms because they provide encrypted communication and certificate-based identity verification. This distinction is important because the dissertation evaluates measurable Kafka transport controls rather than claiming to implement a full system-level Zero Trust deployment.

The TLS and mTLS configurations permit TLS 1.3 and TLS 1.2, with protocol negotiation handled by the Kafka and JDK SSL stack. This reflects the implemented artefact configuration and avoids assuming that all benchmarked connections use TLS 1.3 only.

Mode	Description
Plaintext	Baseline Kafka transport with no TLS encryption.
TLS	Encrypted Kafka transport with server-side certificate authentication.
mTLS	Encrypted Kafka transport with mutual certificate authentication between clients and brokers.

Table B.3: Security modes used by the artefact

B.4 Validity Constraints

Not every possible parameter combination is valid in Kafka. The plan generation process therefore applies validity constraints before benchmark execution. For example, the replication factor cannot exceed the number of available brokers, and the minimum in-sync replica setting must be valid for the selected replication factor.

These constraints are applied before the generated JSONL plan is executed. This prevents invalid benchmark rows from being submitted to the cluster and ensures that the final plan represents executable Kafka configurations rather than a purely theoretical parameter grid.

The use of deterministic run identifiers also supports traceability. Each run identifier is derived from the workload configuration, including security mode, broker count, replication factor, minimum in-sync replicas, message size, target throughput, batch size, acknowledgement mode, producer count, compression type, and trial index. This makes it possible to link each plan row to its result directory and summary output.

For example, a generated run directory may include a full configuration-derived identifier such as `security-overhead-final-factorial-mtls-b5-rf5-isr3-msg102400-tps10000-batch131072-acks1-prod12-complz4-trial3`. This long identifier is intentional because it makes the run self-describing and reduces ambiguity when inspecting result directories.

B.5 Consumer Validation Slice

The consumer-side benchmark is intentionally designed as a targeted validation slice rather than a full factorial campaign. A full consumer factorial campaign would greatly increase the experimental cost and runtime, while duplicating much of the producer-side parameter space. The purpose of the consumer slice is therefore narrower: to test whether secure transport also affects Kafka's read path under selected workloads.

The consumer configuration is defined by `config/factorials/consumer-security-slice.json`. The generated consumer execution plan is stored as `.orchestration/consumer-security-slice-plan.jsonl`. This plan includes plaintext, TLS, and mTLS modes, selected

message sizes, compression settings, consumer concurrency, and repeated trials.

The consumer validation results are therefore used as supporting evidence rather than as a second full factorial evaluation. This keeps the dissertation focused on the main producer-side security-overhead campaign while still addressing the possibility that TLS and mTLS affect consumers as well as producers and brokers.

Appendix C

Result Schema

This appendix summarises the structure of the benchmark outputs produced by the artefact. Each benchmark run produces a run directory containing raw benchmark logs, workload metadata, parsed metrics, and host telemetry. The exact metric values differ between runs, but the output structure remains stable across plaintext, TLS, and mTLS modes.

The result schema was designed to support traceability from experiment plan to dissertation output. Each run can be linked back to its configuration row, security mode, workload parameters, raw Kafka benchmark output, parsed result file, host telemetry files, and aggregated summary row. This makes the result pipeline reproducible and reduces the risk of interpreting a metric without its workload context.

C.1 Run-Level Output

Each completed benchmark run is stored in a dedicated result directory. The directory name is derived from the run identifier, which encodes the main workload parameters. This makes the result directory self-describing and allows individual runs to be inspected without relying only on the aggregated CSV output.

The most important run-level files are `metadata.json`, `result.json`, the raw Kafka benchmark logs, and the `host-telemetry/` directory. The metadata file records the configuration used for the run, while the result file stores parsed benchmark metrics and run-level telemetry summaries.

File or directory	Description
`metadata.json`	Stores workload, deployment, and security-mode configuration for the run.
`result.json`	Stores parsed application metrics and aggregated telemetry fields for the run.
`producer-perf.log`	Stores raw output from the Kafka producer performance benchmark.
`consumer-perf.log`	Stores raw output from the Kafka consumer performance benchmark, where consumer benchmarking is used.
`topic-create.log`	Stores topic creation output for the run.
`topic-delete.log`	Stores topic deletion output where per-run cleanup is used.
`host-telemetry/`	Stores per-host telemetry logs collected from the benchmark client and Kafka brokers.

Table C.1: Run-level benchmark output files

C.2 Metadata Fields

The `metadata.json` file records the workload and deployment context for a benchmark run. This includes the security mode, broker count, partition count, replication factor, minimum in-sync replicas, message size, record count, target throughput, producer count, consumer count, batch size, acknowledgement setting, compression type, and trial index.

These fields are important because throughput and latency are only meaningful when interpreted alongside the workload that produced them. For example, a latency value from a plaintext run with one producer and small messages is not directly comparable with a mTLS run using twelve producers, larger messages, and stricter durability settings unless the configuration is visible.

The metadata fields also support matched comparison across security modes. Plaintext, TLS, and mTLS rows can be grouped using the same workload parameters, allowing the evaluation to isolate transport-security overhead from unrelated workload differences.

Metadata field group	Examples
Security configuration	`security_mode`, client configuration path, listener protocol.
Cluster configuration	`broker_count`, `partition_count`, `replication_factor`, `min_insync_replicas`.
Producer workload	`message_size_bytes`, `num_records`, `target_messages_per_second`, `producer_count`.
Producer tuning	`batch_size`, `linger_ms`, `acks`, `compression_type`.
Consumer workload	`consumer_count` and consumer benchmark settings where applicable.
Run identity	`run_id`, `trial_index`, `trial_count`, and result directory.

Table C.2: Metadata field groups

C.3 Parsed Result Fields

The `result.json` file stores parsed metrics extracted from the Kafka benchmark output. For producer runs, this includes throughput in records per second, throughput in MB per second, average latency, maximum latency, and other latency fields reported by the Kafka tool where available. For consumer runs, the parsed output records consumer throughput and read-path metrics where available.

The result file also stores workload metadata and telemetry summaries so that each run remains interpretable even outside the aggregated result set. This is useful when inspecting individual heavy runs, failed runs, or outliers in the final analysis.

The primary application-level metrics used in the dissertation are throughput in records per second, throughput in MB per second, average latency, and maximum latency. These metrics are used because they are directly reported by the Kafka benchmark tools and provide a consistent basis for comparison across plaintext, TLS, and mTLS runs.

Metric	Description
Throughput records/sec	Number of records successfully sent or consumed per second.
Throughput MB/sec	Data throughput converted into megabytes per second.
Average latency	Mean producer latency reported by the Kafka producer benchmark.
Maximum latency	Maximum producer latency reported by the Kafka producer benchmark.
Consumer throughput	Consumer-side throughput for targeted read-path validation runs.
Failure status	Indicates failed or timed-out runs where applicable.

Table C.3: Primary parsed result metrics

C.4 Host Telemetry Fields

Host telemetry is stored under the `host-telemetry/` directory for each run. The artefact collects telemetry from the benchmark client and Kafka brokers during benchmark execution. These files are stored as JSONL logs, with each line representing one interval sample from a host.

The telemetry captures CPU utilisation, memory utilisation, disk activity, and network receive/transmit counters. These measurements are used to link application-level behaviour to host-level resource use. This is important for the dissertation because TLS and mTLS overhead may appear not only as lower throughput or higher latency, but also as increased CPU activity or different network and disk behaviour.

The number of telemetry hosts is also recorded. For a five-broker campaign, the expected telemetry host count is normally six: five Kafka brokers and one benchmark client. This helps identify whether a run has complete host-level coverage.

Telemetry field group	Description
CPU utilisation	Per-host CPU usage sampled during the benchmark run.
Memory utilisation	Per-host memory usage during the benchmark run.
Network activity	Network receive and transmit counters for each telemetry host.
Disk activity	Disk read and write activity, including sector counters where available.
Telemetry host count	Number of hosts with telemetry files for the run.
Telemetry role	Identifies whether the host is a broker or benchmark client.

Table C.4: Host telemetry field groups

C.5 Result-Set Output

At the result-set level, completed runs are aggregated into `summary.json` and `summary.csv`. The JSON file preserves structured run data, while the CSV file provides a flat format suitable for statistical analysis, plotting, and dissertation tables.

The executor also writes `started.jsonl`, `completed.jsonl`, and `failures.jsonl` where applicable. These files record execution state. The `started.jsonl` file records rows that began execution, while `completed.jsonl` records rows that finished successfully and were copied back locally. The `failures.jsonl` file is only produced when failures occur.

This distinction matters because a started row is not necessarily a completed row. A run is treated as complete only when the benchmark has finished, the result files have been copied back, the raw output has been parsed, and the checkpoint has been updated.

Result-set file	Description
`started.jsonl`	Records benchmark rows that were started by the executor.
`completed.jsonl`	Records benchmark rows that completed successfully.
`failures.jsonl`	Records failed or timed-out rows when failures occur.
`summary.json`	Stores structured summary data across completed runs.
`summary.csv`	Stores flat tabular summary data for analysis and export.
`export/table.csv`	Dissertation-ready CSV table output where export is enabled.
`export/table.tex`	Dissertation-ready LaTeX table output where export is enabled.
`export/*.svg`	Plot outputs where export is enabled.

Table C.5: Result-set output files

C.6 Metric Interpretation Caveat

The benchmark tools provide application-level throughput and latency metrics, while the telemetry pipeline provides interval-based host-resource measurements. These two sources are complementary but not identical. Application metrics describe benchmark behaviour, while telemetry fields describe resource behaviour sampled during the run.

Where interval-derived percentile fields are present in the summary output, they are treated as diagnostic indicators rather than true per-record latency percentiles. They describe variation across sampled intervals or parsed benchmark summaries and are therefore used to support interpretation, not as a substitute for full per-record latency tracing.

For this reason, the dissertation focuses on metrics that are consistently available across runs: throughput records per second, throughput MB per second, average latency, maximum latency, CPU utilisation, memory utilisation, network activity, disk activity, telemetry host count, workload metadata, and failure status.

Appendix D

Additional Result Tables

Appendix E

Repository and Reproducibility Notes

This appendix summarises how the artefact repository is organised and how reproducibility is supported. The repository separates infrastructure provisioning, Kafka deployment, benchmark execution, telemetry collection, result parsing, and export. This was done so that the artefact can be inspected as a set of clear components rather than as one large script.

The artefact is also designed around explicit configuration files and generated execution plans. This means that benchmark rows are not created manually during execution. Instead, the configuration files define the experiment space, the plan generator expands this into JSONL rows, and the executor runs those rows while recording started, completed, and failed runs.

E.1 Repository Structure

The main repository directories are shown in Table E.1. Source files, configuration files, orchestration scripts, and documentation are part of the artefact. Generated execution state is stored separately under `.orchestration/`, because it depends on the current AWS deployment and local execution environment.

Repository path	Purpose
`infrastructure/terraform/`	Contains Terraform configuration for AWS networking, security groups, Kafka broker instances, and the benchmark client.
`deploy/kafka/`	Contains Kafka installation scripts, broker configuration templates, client configuration, TLS/mTLS setup, and benchmark runner scripts.
`config/baselines/`	Contains baseline workload configurations used as fixed defaults.
`config/sweeps/`	Contains one-factor-at-a-time exploratory sweep definitions.
`config/factorials/`	Contains factorial experiment definitions for the final producer campaign and consumer validation slice.
`scripts/orchestration/`	Contains scripts for inventory generation, deployment, benchmark execution, parsing, aggregation, export, and campaign control.
`docs/`	Contains supporting artefact documentation, including architecture, methodology, result schema, reproducibility notes, and campaign operating notes.
`results/`	Contains local benchmark outputs, parsed results, summaries, and exported tables or plots.
`.orchestration/`	Contains generated local execution state, plans, checkpoints, Terraform outputs, inventory files, and deployment metadata.

Table E.1: Repository structure

E.2 Configuration-Driven Execution

Benchmark execution is driven by configuration files rather than manually constructed commands. Baseline and factorial JSON files define the workload parameters, while generated JSONL plans define the exact benchmark rows to execute. This makes the campaign design inspectable before execution and reduces the risk of changing parameters accidentally between security modes.

The final producer campaign is generated from `config/factorials/security-overhead-final.json`. The targeted consumer validation slice is generated from `config/factorials/consumer-security-slice.json`. The generated plans are stored under `.orchestration/`, including `.orchestration/security-overhead-final-plan.jsonl` and `.orchestration/consumer-security-slice-plan.jsonl`.

This structure supports reproducibility because each result can be traced back to a configuration file, generated plan row, deterministic run identifier, metadata file, result directory, and summary row. A benchmark result is therefore not only a value in a table,

but an output linked to a specific workload and deployment configuration.

E.3 Deployment Reproducibility

AWS infrastructure is provisioned using Terraform. The Terraform layer defines the VPC, subnet, security groups, Kafka broker instances, and benchmark client instance. After provisioning, orchestration scripts export the Terraform outputs and convert them into local inventory and cluster metadata files.

Kafka deployment is automated through shell scripts. These scripts install Kafka, render broker configuration, configure KRaft metadata, start broker services, and validate readiness. Kafka data-plane traffic uses private VPC addresses, while SSH orchestration uses public addresses. This separation prevents Kafka benchmark traffic from using public broker addresses while still allowing the local workstation to manage the cluster.

Broker readiness is checked using Kafka API availability rather than only `systemd` process status. This is important because a broker process can be running before the broker is ready to serve Kafka requests. API-level readiness checks reduce the risk of starting benchmark rows against a partially available cluster.

E.4 Checkpointing and Resumability

The benchmark campaigns are long-running and can be interrupted by SSH timeouts, local machine sleep, network issues, or cloud-instance disruption. The executor therefore records campaign state using `started.jsonl`, `completed.jsonl`, `failures.jsonl` where applicable, and checkpoint files under `.orchestration/`.

A row is only treated as complete after the remote benchmark has finished, result files have been copied back locally, raw output has been parsed, and the checkpoint has been updated. This means that a row appearing in `started.jsonl` is not enough for it to be treated as complete. The completed file and checkpoint are the authoritative record of successful execution.

When a campaign resumes, the executor skips rows that are already recorded as complete. This prevents completed rows from being rerun and makes it practical to execute large campaigns across multiple sessions. It also protects the result set from partial local state, because result copying is completed before a row is marked as finished.

E.5 Result Traceability

Each benchmark run writes a dedicated result directory. The directory name is derived from the run identifier, which encodes the main workload configuration. This makes the directory self-describing and reduces ambiguity when inspecting individual runs.

The run directory contains raw Kafka benchmark logs, workload metadata, parsed metrics, and host telemetry. The raw logs are retained so that parsed values can be checked against the original benchmark output. The `metadata.json` file records workload context, while `result.json` stores the parsed structured result for the run.

At result-set level, `summary.json` and `summary.csv` aggregate completed runs into structured and tabular formats. These files are used for analysis, comparison, and dissertation-ready exports. This gives two levels of traceability: individual run inspection through run directories, and full result-set analysis through summary files.

E.6 Generated State and Version Control

Not every file produced by the artefact is suitable for version control. Generated state under `.orchestration/` may include environment-specific inventory files, Terraform outputs, deployment checkpoints, generated plans, SSH key paths, certificates, and cluster metadata. These files are useful for local execution but do not represent permanent source code.

Sensitive files such as SSH keys, AWS credentials, generated certificates, keystores, truststores, and environment-specific deployment outputs are excluded from version control where appropriate. This is necessary for security and also improves reproducibility, because those files describe one live deployment rather than the general artefact design.

Large raw result sets are also treated carefully. Historical result sets are retained locally for traceability, but final analysis should use the telemetry-enabled result sets from the final plaintext, TLS, and mTLS campaigns. Older non-telemetry outputs are useful for development history but should not be mixed with final evaluation data.

E.7 Rerunning the Artefact

To rerun the artefact, the infrastructure must first be provisioned through Terraform. Terraform outputs are then exported into local orchestration metadata. The benchmark client and Kafka brokers are prepared using the orchestration scripts, after which the required security mode is deployed.

For factorial campaigns, the relevant JSON configuration is converted into a JSONL

execution plan. The executor then runs the plan using security-mode and broker-count filters where required. If execution is interrupted, the same command can be rerun with the same checkpoint file so that completed rows are skipped.

The exact command depends on the campaign phase and security mode. However, the reproducibility principle remains the same across all phases: the same configuration file, generated plan, result-set name, and checkpoint file should be used when resuming a campaign.

E.8 Reproducibility Limitations

The artefact improves reproducibility, but it does not remove all sources of experimental variation. AWS EC2 instances are shared cloud resources, and their performance can vary because of cloud scheduling, network conditions, and background host activity. The benchmark results should therefore be interpreted as controlled cloud measurements rather than deterministic hardware constants.

The generated execution state also depends on campaign history. For example, an interrupted run may appear in `started.jsonl` without appearing in `completed.jsonl`. This is expected behaviour and is handled by the executor, but it means that completed rows and checkpoint files should be used when reporting final campaign progress.

Finally, the artefact evaluates selected Kafka transport-security configurations rather than every possible secure Kafka deployment. Plaintext, TLS, and mTLS provide a controlled basis for comparing transport-security overhead, but they do not represent every possible Zero Trust Architecture, certificate-management strategy, authorisation policy, or production Kafka deployment pattern.